

Curso Profesional de Entity Framework Core 8

Módulo 4 - Optimización y rendimiento

PRÁCTICAS

AUTOR: JAIME GALLO

.NET 8 · Entity Framework Core 8 · Visual Studio Community · SQL Server LocalDB · Proyecto AceriaData

Índice

Punto 4.1 — Análisis del SQL generado: ToQueryString y logging	3
Punto 4.2 — Tracking y No Tracking	9
Punto 4.3 — AsNoTracking y AsNoTrackingWithIdentityResolution	15
Punto 4.4 — Problema N+1: identificación y causas	21
Punto 4.5 — Solución a N+1: Include, proyecciones y Split Queries	27
Punto 4.6 — Over-fetching: causas y soluciones	33
Punto 4.7 — Consultas ineficientes: traducción y frontera cliente/servidor	39
Punto 4.8 — Split Queries: cuándo y cómo usarlas	46
Punto 4.9 — Compiled Queries	52
Punto 4.10 — Paginación eficiente: Skip/Take y keyset pagination	59
Punto 4.11 — Diagnóstico con logs, métricas y herramientas	67
Punto 4.12 — Estrategias de optimización y checklist de rendimiento	73

Curso Profesional de Entity Framework Core 8

Módulo 4 — Prácticas: Optimización y rendimiento

Autor: JAIME GALLO

Cada práctica trabaja sobre un checkpoint completo. La secuencia es acumulativa desde M03/PROYECTO/3.12.

Punto 4.1 — Análisis del SQL generado: ToQueryString y logging

Contexto del proyecto

Este checkpoint continúa M03/PROYECTO/3.12. Conserva solución, capas, filtros, índices y migraciones; M4 no crea migraciones vacías.

Objetivo práctico

ToQueryString inspecciona la representación SQL sin materializar; el logging muestra los comandos realmente ejecutados. Son herramientas complementarias y el manual definitivo usa el checkpoint validado.

Paso 1: Abrir el checkpoint

```
cd M04/PROYECTO/4.1
dotnet restore AceriaData.sln
```

POWERSHELL

Paso 2: Comprobar migraciones

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

POWERSHELL

Debe seguir apareciendo M2_2_12_Architecture.

Paso 3: Identificar el delta docente

El archivo principal del delta es src/AceriaData.Infrastructure/Repositories/Rendimiento41.cs. El checkpoint conserva todo el estado anterior.

Paso 4: Implementar y estudiar Infrastructure

```
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public sealed partial class OrdenRepositorio
{
    public string ObtenerSqlPendientesOrdenadasM4() => _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Estado == "Pendiente")
        .OrderBy(o => o.FechaCreacion)
        .ThenBy(o => o.Id)
        .ToQueryString();

    public string ObtenerSqlConIncludeM4() => _context.OrdenesFabricacion
        .AsNoTracking()
        .Include(o => o.Planchas)
        .Where(o => o.Cliente == "Constructora del Norte")
}
```

CSHARP

```

        .OrderBy(o => o.Id)
        .ToQueryString();

    public string ObtenerSqlConProyeccionM4() => _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Estado == "Pendiente")
        .OrderBy(o => o.FechaCreacion)
        .ThenBy(o => o.Id)
        .Select(o => new { o.NumeroOrden, o.Cliente })
        .ToQueryString();
}

```

Archivo complementario: src/AceriaData.Infrastructure/DependencyInjection.cs

```

using AceriaData.Application.Interfaces;
using AceriaData.Infrastructure.Persistence;
using AceriaData.Infrastructure.Repositories;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.Infrastructure;

public static class DependencyInjection
{
    public static IServiceCollection AddAceriaInfrastructure(this IServiceCollection services, string
connectionString)
    {
        services.AddDbContext<AceriaDbContext>(o => o
            .UseSqlServer(connectionString)
            .EnableDetailedErrors()
            .LogTo(
                Console.WriteLine,
                new[] { DbLoggerCategory.Database.Command.Name },
                LogLevel.Information));
        services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IUnidadDeTrabajo, UnidadDeTrabajo>();
        return services;
    }
}

```

CSHARP

Application no recibe DbContext, IQueryable ni referencias a Microsoft.EntityFrameworkCore.

Paso 5: Implementar el caso de uso

```

using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class AnalisisSqlUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public AnalisisSqlUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== 4.1 ANALISIS DEL SQL GENERADO ===");
        var pendientes = _unidad.Ordenes.ObtenerSqlPendientesOrdenadasM4();
        var include = _unidad.Ordenes.ObtenerSqlConIncludeM4();
        var proyeccion = _unidad.Ordenes.ObtenerSqlConProyeccionM4();

        if (!pendientes.Contains("WHERE", StringComparison.OrdinalIgnoreCase))
            throw new InvalidOperationException("4.1: el SQL de pendientes no contiene filtro.");
        if (!include.Contains("JOIN", StringComparison.OrdinalIgnoreCase))
            throw new InvalidOperationException("4.1: el SQL con Include no contiene JOIN.");
        if (proyeccion.Contains("Observaciones", StringComparison.OrdinalIgnoreCase))
            throw new InvalidOperationException("4.1: la proyeccion recupera columnas no solicitadas.");

        Console.WriteLine("--- SQL pendientes ---");
        Console.WriteLine(pendientes);
        Console.WriteLine("--- SQL Include ---");
        Console.WriteLine(include);
        Console.WriteLine("--- SQL proyeccion ---");
        Console.WriteLine(proyeccion);
    }
}

```

CSHARP

Explicación línea a línea — AnalisisSqlUseCase.cs

Línea 1	<code>using AceriaData.Application.Interfaces;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	<code>` `</code> → Separa bloques lógicos.
Línea 3	<code>namespace AceriaData.Application.UseCases;</code> → Sitúa el archivo en la capa y espacio de nombres correspondiente.
Línea 4	<code>` `</code> → Separa bloques lógicos.
Línea 5	<code>public sealed class AnalisisSqlUseCase</code> → Declara la clase concreta usada por el checkpoint.
Línea 6	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 7	<code>private readonly IUnidadDeTrabajo _unidad;</code> → Declara la dependencia conservada por la instancia.
Línea 8	<code>public AnalisisSqlUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;</code> → Constructor del caso de uso e inyección de la unidad de trabajo.
Línea 9	<code>` `</code> → Separa bloques lógicos.
Línea 10	<code>public void Ejecutar()</code> → Define el flujo principal validado por el E2E.
Línea 11	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 12	<code>Console.WriteLine("=== 4.1 ANALISIS DEL SQL GENERADO ===");</code> → Publica evidencia observable en la consola.
Línea 13	<code>var pendientes = _unidad.Ordenes.ObtenerSqlPendientesOrdenadasM4();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 14	<code>var include = _unidad.Ordenes.ObtenerSqlConIncludeM4();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 15	<code>var proyeccion = _unidad.Ordenes.ObtenerSqlConProyeccionM4();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 16	<code>` `</code> → Separa bloques lógicos.
Línea 17	<code>if (!pendientes.Contains("WHERE", StringComparison.OrdinalIgnoreCase))</code> → Comprueba una condición contractual del E2E.
Línea 18	<code>throw new InvalidOperationException("4.1: el SQL de pendientes no contiene filtro.");</code> → Hace fallar el checkpoint si la evidencia no coincide.
Línea 19	<code>if (!include.Contains("JOIN", StringComparison.OrdinalIgnoreCase))</code> → Comprueba una condición contractual del E2E.
Línea 20	<code>throw new InvalidOperationException("4.1: el SQL con Include no contiene JOIN.");</code> → Hace fallar el checkpoint si la evidencia no coincide.
Línea 21	<code>if (proyeccion.Contains("Observaciones", StringComparison.OrdinalIgnoreCase))</code> → Comprueba una condición contractual del E2E.
Línea 22	<code>throw new InvalidOperationException("4.1: la proyeccion recupera columnas no solicitadas.");</code> → Hace fallar el checkpoint si la evidencia no coincide.
Línea 23	<code>` `</code> → Separa bloques lógicos.
Línea 24	<code>Console.WriteLine("--- SQL pendientes ---");</code> → Publica evidencia observable en la consola.

Línea 25	Console.WriteLine(pendientes); → Publica evidencia observable en la consola.
Línea 26	Console.WriteLine("--- SQL Include ---"); → Publica evidencia observable en la consola.
Línea 27	Console.WriteLine(include); → Publica evidencia observable en la consola.
Línea 28	Console.WriteLine("--- SQL proyeccion ---"); → Publica evidencia observable en la consola.
Línea 29	Console.WriteLine(proyeccion); → Publica evidencia observable en la consola.
Línea 30	} → Delimita el bloque sintáctico asociado.
Línea 31	} → Delimita el bloque sintáctico asociado.

Paso 6: Preparar el composition root

CSHARP

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<BuenasPracticasUseCase>();
services.AddScoped<AnalisisSqlUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,
    ValidateScopes = true
});
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<AnalisisSqlUseCase>();
useCase.Ejecutar();

Console.WriteLine("4.1 OK");
```

Explicación línea a línea — Program.cs

Línea 1	using AceriaData.Application.UseCases; → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	using AceriaData.ConsoleApp; → Importa tipos o extensiones requeridos por esta implementación.
Línea 3	using AceriaData.Infrastructure; → Importa tipos o extensiones requeridos por esta implementación.
Línea 4	using AceriaData.Infrastructure.Persistence; → Importa tipos o extensiones requeridos por esta implementación.

Línea 5	<code>using Microsoft.EntityFrameworkCore; → Importa tipos o extensiones requeridos por esta implementación.</code>
Línea 6	<code>using Microsoft.Extensions.Configuration; → Importa tipos o extensiones requeridos por esta implementación.</code>
Línea 7	<code>using Microsoft.Extensions.DependencyInjection; → Importa tipos o extensiones requeridos por esta implementación.</code>
Línea 8	<code>` ` → Separa bloques lógicos.</code>
Línea 9	<code>var configuration = new ConfigurationBuilder() → Calcula y conserva el resultado que será validado o mostrado.</code>
Línea 10	<code>.SetBasePath(AppContext.BaseDirectory) → Participa directamente en el flujo validado del checkpoint: .SetBasePath(AppContext.BaseDirectory)</code>
Línea 11	<code>.AddJsonFile("appsettings.json", optional: false) → Participa directamente en el flujo validado del checkpoint: .AddJsonFile("appsettings.json", optional: false)</code>
Línea 12	<code>.AddEnvironmentVariables() → Participa directamente en el flujo validado del checkpoint: .AddEnvironmentVariables()</code>
Línea 13	<code>.Build(); → Participa directamente en el flujo validado del checkpoint: .Build();</code>
Línea 14	<code>` ` → Separa bloques lógicos.</code>
Línea 15	<code>var connectionString = configuration.GetConnectionString("AceriaDB") → Calcula y conserva el resultado que será validado o mostrado.</code>
Línea 16	<code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB"); → Participa directamente en el flujo validado del checkpoint: ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");</code>
Línea 17	<code>` ` → Separa bloques lógicos.</code>
Línea 18	<code>var services = new ServiceCollection(); → Calcula y conserva el resultado que será validado o mostrado.</code>
Línea 19	<code>services.AddAceriaInfrastructure(connectionString); → Participa directamente en el flujo validado del checkpoint: services.AddAceriaInfrastructure(connectionString);</code>
Línea 20	<code>services.AddScoped<BuenasPracticasUseCase>(); → Registra el caso de uso con ciclo de vida Scoped.</code>
Línea 21	<code>services.AddScoped<AnalisisSqlUseCase>(); → Registra el caso de uso con ciclo de vida Scoped.</code>
Línea 22	<code>` ` → Separa bloques lógicos.</code>
Línea 23	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions → Importa tipos o extensiones requeridos por esta implementación.</code>
Línea 24	<code>{ → Delimita el bloque sintáctico asociado.</code>
Línea 25	<code>ValidateOnBuild = true, → Participa directamente en el flujo validado del checkpoint: ValidateOnBuild = true,</code>
Línea 26	<code>ValidateScopes = true → Participa directamente en el flujo validado del checkpoint: ValidateScopes = true</code>
Línea 27	<code>}); → Participa directamente en el flujo validado del checkpoint: });</code>

Línea 28	<code>using var scope = provider.CreateScope();</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 29	<code>` `</code> → Separa bloques lógicos.
Línea 30	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve una dependencia obligatoria desde el ámbito.
Línea 31	<code>context.Database.EnsureDeleted();</code> → Reinicia la base de demostración para un E2E determinista.
Línea 32	<code>context.Database.Migrate();</code> → Aplica la historia real de migraciones heredada.
Línea 33	<code>DemoData.Seed(context);</code> → Carga el dataset determinista de <code>AceriaData</code> .
Línea 34	<code>context.ChangeTracker.Clear();</code> → Limpia tracking antes de la demostración.
Línea 35	<code>` `</code> → Separa bloques lógicos.
Línea 36	<code>var useCase = scope.ServiceProvider.GetRequiredService<AnálisisSqlUseCase>();</code> → Resuelve una dependencia obligatoria desde el ámbito.
Línea 37	<code>useCase.Ejecutar();</code> → Participa directamente en el flujo validado del checkpoint: <code>useCase.Ejecutar();</code>
Línea 38	<code>` `</code> → Separa bloques lógicos.
Línea 39	<code>Console.WriteLine("4.1 OK");</code> → Publica evidencia observable en la consola.

Paso 7: Compilar

```
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

El build debe finalizar sin errores; el delta se propaga a los estados posteriores.

Paso 8: Ejecutar en LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

POWERSHELL

La salida debe terminar con 4.1 OK. Las aserciones internas fallan si la evidencia no coincide.

Paso 9: Diagnóstico técnico

Reto resuelto. Comparar el SQL de una entidad completa con el de una proyección y justificar qué columnas sobran.

No uses solo tiempo: revisa SQL/roundtrips, cardinalidad, columnas, tracking y frontera de materialización.

Paso 10: Cierre acumulativo

1. Confirma el marcador E2E.
2. Comprueba que `EnsureCreated` no aparece.
3. Conserva las migraciones heredadas.
4. Verifica que `Application` no depende de EF Core.
5. Compara con el checkpoint anterior y documenta el delta.

Errores comunes revisados

- Confundir una medición aislada con una conclusión de rendimiento.
- Materializar antes de terminar filtros o proyecciones sin intención.
- Aplicar una técnica por regla general en lugar de observar la consulta.

- Relajar una aserción para ocultar un fallo en vez de corregir su causa.

Trazabilidad con la práctica fuente

La práctica fuente de 4.1 incluía además los siguientes focos docentes:

- ToQueryString antes de materializar y logging de comandos SQL.
- Consultas con Where, OrderBy, Select e Include, incluyendo el filtro global de Soft Delete.
- Análisis de múltiples Include como origen potencial de multiplicación de filas.

Tratamiento en el M4 definitivo. El checkpoint valida ToQueryString, logging, filtro, Include y proyección. El reto de múltiples colecciones se conserva como puente hacia 4.8, donde se demuestra con dos colecciones reales.

Reto de ampliación procedente de la fuente. Construye mentalmente una consulta con dos colecciones incluidas y anticipa cómo crecerían las filas; compruébalo después en 4.8.

Errores de la fuente que deben seguir siendo diagnosticables

- No materializar con ToList antes de pedir ToQueryString.
- No exponer IQueryable desde Application.
- No resolver servicios Scoped desde el proveedor raíz.

Analogía operativa

ToQueryString es el plano previo; el logging es el registro de lo que realmente pasó por la línea.

Resultado esperado

El checkpoint 4.1 queda ejecutable, trazado y reproducible, y sirve como baseline física del punto siguiente.

Conexión con el siguiente punto

El siguiente estado es 4.2 y parte físicamente de este checkpoint.

Punto 4.2 — Tracking y No Tracking

Contexto del proyecto

Este checkpoint continúa M04/PROYECTO/4.1. Conserva solución, capas, filtros, índices y migraciones; M4 no crea migraciones vacías.

Objetivo práctico

Tracking y NoTracking normalmente no cambian el SELECT: cambian sobre todo materialización y ChangeTracker. Un DTO puro sin entidades no se rastrea; una proyección que contenga entidades sí puede mantener tracking de esas entidades.

Paso 1: Abrir el checkpoint

```
cd M04/PROYECTO/4.2
dotnet restore AceriaData.sln
```

POWERSHELL

Paso 2: Comprobar migraciones

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

POWERSHELL

Debe seguir apareciendo M2_2_12_Architecture.

Paso 3: Identificar el delta docente

El archivo principal del delta es src/AceriaData.Infrastructure.Repositories/Rendimiento42.cs. El checkpoint conserva todo el estado anterior.

Paso 4: Implementar y estudiar Infrastructure

```
using AceriaData.Application.Dtos;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public sealed partial class OrdenRepository
{
    public TrackingMetricaDto MedirConsultaConTrackingM4()
    {
        _context.ChangeTracker.Clear();
        var consulta = _context.OrdenesFabricacion
            .AsTracking()
            .OrderBy(o => o.Id);
        var sql = consulta.ToQueryString();
        var filas = consulta.ToList().Count;
        return new TrackingMetricaDto
        {
            Filas = filas,
            EntidadesRastreadas = _context.ChangeTracker.Entries().Count(),
            Sql = sql
        };
    }

    public TrackingMetricaDto MedirConsultaSinTrackingM4()
    {
        _context.ChangeTracker.Clear();
        var consulta = _context.OrdenesFabricacion
            .AsNoTracking()
            .OrderBy(o => o.Id);
        var sql = consulta.ToQueryString();
        var filas = consulta.ToList().Count;
        return new TrackingMetricaDto
        {
            Filas = filas,
            EntidadesRastreadas = _context.ChangeTracker.Entries().Count(),
            Sql = sql
        };
    }
}
```

CSHARP

Application no recibe DbContext, IQueryable ni referencias a Microsoft.EntityFrameworkCore.

Paso 5: Implementar el caso de uso

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class TrackingUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public TrackingUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== 4.2 TRACKING Y NO TRACKING ===");
        var con = _unidad.Ordenes.MedirConsultaConTrackingM4();
        var sin = _unidad.Ordenes.MedirConsultaSinTrackingM4();

        if (con.Filas == 0 || con.EntidadesRastreadas != con.Filas)
            throw new InvalidOperationException("4.2: tracking no produjo el numero esperado de entradas.");
        if (sin.Filas != con.Filas || sin.EntidadesRastreadas != 0)
            throw new InvalidOperationException("4.2: AsNoTracking dejo entidades rastreadas.");

        Console.WriteLine($"Con tracking: filas={con.Filas}, rastreadas={con.EntidadesRastreadas}");
        Console.WriteLine($"Sin tracking: filas={sin.Filas}, rastreadas={sin.EntidadesRastreadas}");
        Console.WriteLine("El SQL puede ser equivalente; la diferencia relevante esta en la materializacion y el ChangeTracker.");
    }
}
```

CSHARP

Explicación línea a línea — TrackingUseCase.cs

Línea 1	<code>using AceriaData.Application.Interfaces;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	<code>` `</code> → Separa bloques lógicos.
Línea 3	<code>namespace AceriaData.Application.UseCases;</code> → Sitúa el archivo en la capa y espacio de nombres correspondiente.
Línea 4	<code>` `</code> → Separa bloques lógicos.
Línea 5	<code>public sealed class TrackingUseCase</code> → Declara la clase concreta usada por el checkpoint.
Línea 6	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 7	<code>private readonly IUnidadDeTrabajo _unidad;</code> → Declara la dependencia conservada por la instancia.
Línea 8	<code>public TrackingUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;</code> → Constructor del caso de uso e inyección de la unidad de trabajo.
Línea 9	<code>` `</code> → Separa bloques lógicos.
Línea 10	<code>public void Ejecutar()</code> → Define el flujo principal validado por el E2E.
Línea 11	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 12	<code>Console.WriteLine("=== 4.2 TRACKING Y NO TRACKING ===");</code> → Publica evidencia observable en la consola.
Línea 13	<code>var con = _unidad.Ordenes.MedirConsultaConTrackingM4();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 14	<code>var sin = _unidad.Ordenes.MedirConsultaSinTrackingM4();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 15	<code>` `</code> → Separa bloques lógicos.
Línea 16	<code>if (con.Filas == 0 con.EntidadesRastreadas != con.Filas)</code> → Comprueba una condición contractual del E2E.
Línea 17	<code>throw new InvalidOperationException("4.2: tracking no produjo el numero esperado de entradas.");</code> → Hace fallar el checkpoint si la evidencia no coincide.
Línea 18	<code>if (sin.Filas != con.Filas sin.EntidadesRastreadas != 0)</code> → Comprueba una condición contractual del E2E.
Línea 19	<code>throw new InvalidOperationException("4.2: AsNoTracking dejo entidades rastreadas.");</code> → Hace fallar el checkpoint si la evidencia no coincide.
Línea 20	<code>` `</code> → Separa bloques lógicos.
Línea 21	<code>Console.WriteLine(\$"Con tracking: filas={con.Filas}, rastreadas={con.EntidadesRastreadas}");</code> → Publica evidencia observable en la consola.
Línea 22	<code>Console.WriteLine(\$"Sin tracking: filas={sin.Filas}, rastreadas={sin.EntidadesRastreadas}");</code> → Publica evidencia observable en la consola.
Línea 23	<code>Console.WriteLine("El SQL puede ser equivalente; la diferencia relevante esta en la materializacion y el ChangeTracker.");</code> → Publica evidencia observable en la consola.
Línea 24	<code>}</code> → Delimita el bloque sintáctico asociado.

Línea 25 } → Delimita el bloque sintáctico asociado.

Paso 6: Preparar el composition root

CSHARP

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<BuenasPracticasUseCase>();
services.AddScoped<AnalisisSqlUseCase>();
services.AddScoped<TrackingUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true,
    ValidateScopes = true });
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<TrackingUseCase>();
useCase.Ejecutar();

Console.WriteLine("4.2 OK");
```

Explicación línea a línea — Program.cs

Línea 1	using AceriaData.Application.UseCases; → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	using AceriaData.ConsoleApp; → Importa tipos o extensiones requeridos por esta implementación.
Línea 3	using AceriaData.Infrastructure; → Importa tipos o extensiones requeridos por esta implementación.
Línea 4	using AceriaData.Infrastructure.Persistence; → Importa tipos o extensiones requeridos por esta implementación.
Línea 5	using Microsoft.EntityFrameworkCore; → Importa tipos o extensiones requeridos por esta implementación.
Línea 6	using Microsoft.Extensions.Configuration; → Importa tipos o extensiones requeridos por esta implementación.
Línea 7	using Microsoft.Extensions.DependencyInjection; → Importa tipos o extensiones requeridos por esta implementación.
Línea 8	`` → Separa bloques lógicos.
Línea 9	var configuration = new ConfigurationBuilder() → Calcula y conserva el resultado que será validado o mostrado.

Línea 10	<code>.SetBasePath(AppContext.BaseDirectory) → Participa directamente en el flujo validado del checkpoint: .SetBasePath(AppContext.BaseDirectory)</code>
Línea 11	<code>.AddJsonFile("appsettings.json", optional: false) → Participa directamente en el flujo validado del checkpoint: .AddJsonFile("appsettings.json", optional: false)</code>
Línea 12	<code>.AddEnvironmentVariables() → Participa directamente en el flujo validado del checkpoint: .AddEnvironmentVariables()</code>
Línea 13	<code>.Build(); → Participa directamente en el flujo validado del checkpoint: .Build();</code>
Línea 14	<code>` ` → Separa bloques lógicos.</code>
Línea 15	<code>var connectionString = configuration.GetConnectionString("AceriaDB") → Calcula y conserva el resultado que será validado o mostrado.</code>
Línea 16	<code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB"); → Participa directamente en el flujo validado del checkpoint: ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");</code>
Línea 17	<code>` ` → Separa bloques lógicos.</code>
Línea 18	<code>var services = new ServiceCollection(); → Calcula y conserva el resultado que será validado o mostrado.</code>
Línea 19	<code>services.AddAceriaInfrastructure(connectionString); → Participa directamente en el flujo validado del checkpoint: services.AddAceriaInfrastructure(connectionString);</code>
Línea 20	<code>services.AddScoped<BuenasPracticasUseCase>(); → Registra el caso de uso con ciclo de vida Scoped.</code>
Línea 21	<code>services.AddScoped<AnalisisSqlUseCase>(); → Registra el caso de uso con ciclo de vida Scoped.</code>
Línea 22	<code>services.AddScoped<TrackingUseCase>(); → Registra el caso de uso con ciclo de vida Scoped.</code>
Línea 23	<code>` ` → Separa bloques lógicos.</code>
Línea 24	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true, ValidateScopes = true }); → Importa tipos o extensiones requeridos por esta implementación.</code>
Línea 25	<code>using var scope = provider.CreateScope(); → Importa tipos o extensiones requeridos por esta implementación.</code>
Línea 26	<code>` ` → Separa bloques lógicos.</code>
Línea 27	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>(); → Resuelve una dependencia obligatoria desde el ámbito.</code>
Línea 28	<code>context.Database.EnsureDeleted(); → Reinicia la base de demostración para un E2E determinista.</code>
Línea 29	<code>context.Database.Migrate(); → Aplica la historia real de migraciones heredada.</code>
Línea 30	<code>DemoData.Seed(context); → Carga el dataset determinista de AceriaData.</code>
Línea 31	<code>context.ChangeTracker.Clear(); → Limpia tracking antes de la demostración.</code>
Línea 32	<code>` ` → Separa bloques lógicos.</code>
Línea 33	<code>var useCase = scope.ServiceProvider.GetRequiredService<TrackingUseCase>(); → Resuelve una dependencia obligatoria desde el ámbito.</code>

Línea 34	<code>useCase.Ejecutar();</code> → Participa directamente en el flujo validado del checkpoint: <code>useCase.Ejecutar();</code>
Línea 35	<code>` `</code> → Separa bloques lógicos.
Línea 36	<code>Console.WriteLine("4.2 OK");</code> → Publica evidencia observable en la consola.

Paso 7: Compilar

```
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

El build debe finalizar sin errores; el delta se propaga a los estados posteriores.

Paso 8: Ejecutar en LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

POWERSHELL

La salida debe terminar con 4.2 OK. Las aserciones internas fallan si la evidencia no coincide.

Paso 9: Diagnóstico técnico

Reto resuelto. Explicar por qué dos consultas con SQL parecido pueden tener distinto coste de materialización.

No uses solo tiempo: revisa SQL/roundtrips, cardinalidad, columnas, tracking y frontera de materialización.

Paso 10: Cierre acumulativo

1. Confirma el marcador E2E.
2. Comprueba que EnsureCreated no aparece.
3. Conserva las migraciones heredadas.
4. Verifica que Application no depende de EF Core.
5. Compara con el checkpoint anterior y documenta el delta.

Errores comunes revisados

- Confundir una medición aislada con una conclusión de rendimiento.
- Materializar antes de terminar filtros o proyecciones sin intención.
- Aplicar una técnica por regla general en lugar de observar la consulta.
- Relajar una aserción para ocultar un fallo en vez de corregir su causa.

Trazabilidad con la práctica fuente

La práctica fuente de 4.2 incluía además los siguientes focos docentes:

- Tracking, AsTracking, AsNoTracking y coste del ChangeTracker.
- Conteo de entidades rastreadas y comparación aislada entre consultas.
- Tracking de grafos con entidades relacionadas.

Tratamiento en el M4 definitivo. La fuente proponía contextos separados para aislar mediciones. AceriaData usa `ChangeTracker.Clear()` antes de cada escenario, que elimina la contaminación entre mediciones dentro del E2E determinista.

Reto de ampliación procedente de la fuente. Carga un grafo con relaciones con y sin tracking y razona qué entidades quedarían registradas.

Errores de la fuente que deben seguir siendo diagnosticables

- No interpretar SQL idéntico como coste idéntico de materialización.
- No reutilizar estado previo del ChangeTracker al medir.
- No registrar DbContext como Singleton.

Analogía operativa

Tracking es mantener una ficha viva de cada pieza; NoTracking es leerla sin abrir expediente.

Resultado esperado

El checkpoint 4.2 queda ejecutable, trazado y reproducible, y sirve como baseline física del punto siguiente.

Conexión con el siguiente punto

El siguiente estado es 4.3 y parte físicamente de este checkpoint.

Punto 4.3 — AsNoTracking y AsNoTrackingWithIdentityResolution

Contexto del proyecto

Este checkpoint continúa M04/PROYECTO/4.2. Conserva solución, capas, filtros, índices y migraciones; M4 no crea migraciones vacías.

Objetivo práctico

La resolución de identidad solo se demuestra si la misma clave aparece repetida. AceriaData usa Aleacion porque una misma aleación está relacionada con varias órdenes; PlanchaAcero pertenece a una sola orden y no es una evidencia válida.

Paso 1: Abrir el checkpoint

```
cd M04/PROYECTO/4.3
dotnet restore AceriaData.sln
```

POWERSHELL

Paso 2: Comprobar migraciones

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

POWERSHELL

Debe seguir apareciendo M2_2_12_Architecture.

Paso 3: Identificar el delta docente

El archivo principal del delta es src/AceriaData.Infrastructure/Repositories/Rendimiento43.cs. El checkpoint conserva todo el estado anterior.

Paso 4: Implementar y estudiar Infrastructure

```
using AceriaData.Application.Dtos;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public sealed partial class OrdenRepositorio
{
    public IdentityResolutionMetricaDto MedirNoTrackingSinResolucionM4()
    {
        _context.ChangeTracker.Clear();
        var entidades = _context.OrdenesAleaciones
            .AsNoTracking()
            .OrderBy(oa => oa.OrdenFabricacionId)
            .Select(oa => oa.Aleacion)
            .ToList();

        return new IdentityResolutionMetricaDto
    }
}
```

CSHARP

```

    {
        Filas = entidades.Count(),
        ClavesUnicas = entidades.Select(a => a.Id).Distinct().Count(),
        InstanciasUnicas = entidades.Distinct(ReferenceEqualityComparer.Instance).Count()
    };
}

public IdentityResolutionMetricaDto MedirNoTrackingConResolucionM4()
{
    _context.ChangeTracker.Clear();
    var entidades = _context.OrdenesAleaciones
        .AsNoTrackingWithIdentityResolution()
        .OrderBy(oa => oa.OrdenFabricacionId)
        .Select(oa => oa.Aleacion)
        .ToList();

    return new IdentityResolutionMetricaDto
    {
        Filas = entidades.Count(),
        ClavesUnicas = entidades.Select(a => a.Id).Distinct().Count(),
        InstanciasUnicas = entidades.Distinct(ReferenceEqualityComparer.Instance).Count()
    };
}
}

```

Application no recibe DbContext, IQueryable ni referencias a Microsoft.EntityFrameworkCore.

Paso 5: Implementar el caso de uso

CSHARP

```

using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class IdentityResolutionUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public IdentityResolutionUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== 4.3 NO TRACKING E IDENTITY RESOLUTION ===");
        var sin = _unidad.Ordenes.MedirNoTrackingSinResolucionM4();
        var con = _unidad.Ordenes.MedirNoTrackingConResolucionM4();

        if (sin.Filas <= sin.ClavesUnicas)
            throw new InvalidOperationException("4.3: el dataset no contiene una entidad relacionada repetida.");
        if (sin.InstanciasUnicas != sin.Filas)
            throw new InvalidOperationException("4.3: AsNoTracking resolvió identidades cuando no debía.");
        if (con.InstanciasUnicas != con.ClavesUnicas)
            throw new InvalidOperationException("4.3: AsNoTrackingWithIdentityResolution no deduplicó por clave.");

        Console.WriteLine($"AsNoTracking: filas={sin.Filas}, claves={sin.ClavesUnicas}, instancias={sin.InstanciasUnicas}");
        Console.WriteLine($"IdentityResolution: filas={con.Filas}, claves={con.ClavesUnicas}, instancias={con.InstanciasUnicas}");
    }
}

```

Explicación línea a línea — IdentityResolutionUseCase.cs

Línea 1	using AceriaData.Application.Interfaces; → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	`` → Separa bloques lógicos.
Línea 3	namespace AceriaData.Application.UseCases; → Sitúa el archivo en la capa y espacio de nombres correspondiente.
Línea 4	`` → Separa bloques lógicos.
Línea 5	public sealed class IdentityResolutionUseCase → Declara la clase concreta usada por el checkpoint.
Línea 6	{ → Delimita el bloque sintáctico asociado.

Línea 7	<code>private readonly IUnidadDeTrabajo _unidad;</code> → Declara la dependencia conservada por la instancia.
Línea 8	<code>public IdentityResolutionUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;</code> → Constructor del caso de uso e inyección de la unidad de trabajo.
Línea 9	<code>` `</code> → Separa bloques lógicos.
Línea 10	<code>public void Ejecutar()</code> → Define el flujo principal validado por el E2E.
Línea 11	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 12	<code>Console.WriteLine("=== 4.3 NO TRACKING E IDENTITY RESOLUTION ===");</code> → Publica evidencia observable en la consola.
Línea 13	<code>var sin = _unidad.Ordenes.MedirNoTrackingSinResolucionM4();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 14	<code>var con = _unidad.Ordenes.MedirNoTrackingConResolucionM4();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 15	<code>` `</code> → Separa bloques lógicos.
Línea 16	<code>if (sin.Filas <= sin.ClavesUnicas)</code> → Comprueba una condición contractual del E2E.
Línea 17	<code>throw new InvalidOperationException("4.3: el dataset no contiene una entidad relacionada repetida.");</code> → Hace fallar el checkpoint si la evidencia no coincide.
Línea 18	<code>if (sin.InstanciasUnicas != sin.Filas)</code> → Comprueba una condición contractual del E2E.
Línea 19	<code>throw new InvalidOperationException("4.3: AsNoTracking resolvió identidades cuando no debía.");</code> → Hace fallar el checkpoint si la evidencia no coincide.
Línea 20	<code>if (con.InstanciasUnicas != con.ClavesUnicas)</code> → Comprueba una condición contractual del E2E.
Línea 21	<code>throw new InvalidOperationException("4.3: AsNoTrackingWithIdentityResolution no deduplico por clave.");</code> → Hace fallar el checkpoint si la evidencia no coincide.
Línea 22	<code>` `</code> → Separa bloques lógicos.
Línea 23	<code>Console.WriteLine(\$"AsNoTracking: filas={sin.Filas}, claves={sin.ClavesUnicas}, instancias={sin.InstanciasUnicas}");</code> → Publica evidencia observable en la consola.
Línea 24	<code>Console.WriteLine(\$"IdentityResolution: filas={con.Filas}, claves={con.ClavesUnicas}, instancias={con.InstanciasUnicas}");</code> → Publica evidencia observable en la consola.
Línea 25	<code>}</code> → Delimita el bloque sintáctico asociado.
Línea 26	<code>}</code> → Delimita el bloque sintáctico asociado.

Paso 6: Preparar el composition root

CSHARP

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
```

```

.Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<BuenasPracticasUseCase>();
services.AddScoped<AnalisisSqlUseCase>();
services.AddScoped<TrackingUseCase>();
services.AddScoped<IdentityResolutionUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true,
ValidateScopes = true });
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<IdentityResolutionUseCase>();
useCase.Ejecutar();

Console.WriteLine("4.3 OK");

```

Explicación línea a línea — Program.cs

Línea 1	using AceriaData.Application.UseCases; → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	using AceriaData.ConsoleApp; → Importa tipos o extensiones requeridos por esta implementación.
Línea 3	using AceriaData.Infrastructure; → Importa tipos o extensiones requeridos por esta implementación.
Línea 4	using AceriaData.Infrastructure.Persistence; → Importa tipos o extensiones requeridos por esta implementación.
Línea 5	using Microsoft.EntityFrameworkCore; → Importa tipos o extensiones requeridos por esta implementación.
Línea 6	using Microsoft.Extensions.Configuration; → Importa tipos o extensiones requeridos por esta implementación.
Línea 7	using Microsoft.Extensions.DependencyInjection; → Importa tipos o extensiones requeridos por esta implementación.
Línea 8	`` → Separa bloques lógicos.
Línea 9	var configuration = new ConfigurationBuilder() → Calcula y conserva el resultado que será validado o mostrado.
Línea 10	.SetBasePath(AppContext.BaseDirectory) → Participa directamente en el flujo validado del checkpoint: .SetBasePath(AppContext.BaseDirectory)
Línea 11	.AddJsonFile("appsettings.json", optional: false) → Participa directamente en el flujo validado del checkpoint: .AddJsonFile("appsettings.json", optional: false)
Línea 12	.AddEnvironmentVariables() → Participa directamente en el flujo validado del checkpoint: .AddEnvironmentVariables()
Línea 13	.Build(); → Participa directamente en el flujo validado del checkpoint: .Build();
Línea 14	`` → Separa bloques lógicos.
Línea 15	var connectionString = configuration.GetConnectionString("AceriaDB") → Calcula y conserva el resultado que será validado o mostrado.

Línea 16	<code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");</code> → Participa directamente en el flujo validado del checkpoint: <code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");</code>
Línea 17	<code>` `</code> → Separa bloques lógicos.
Línea 18	<code>var services = new ServiceCollection();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 19	<code>services.AddAceriaInfrastructure(connectionString);</code> → Participa directamente en el flujo validado del checkpoint: <code>services.AddAceriaInfrastructure(connectionString);</code>
Línea 20	<code>services.AddScoped<BuenasPracticasUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 21	<code>services.AddScoped<AnalisisSqlUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 22	<code>services.AddScoped<TrackingUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 23	<code>services.AddScoped<IdentityResolutionUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 24	<code>` `</code> → Separa bloques lógicos.
Línea 25	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true, ValidateScopes = true });</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 26	<code>using var scope = provider.CreateScope();</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 27	<code>` `</code> → Separa bloques lógicos.
Línea 28	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve una dependencia obligatoria desde el ámbito.
Línea 29	<code>context.Database.EnsureDeleted();</code> → Reinicia la base de demostración para un E2E determinista.
Línea 30	<code>context.Database.Migrate();</code> → Aplica la historia real de migraciones heredada.
Línea 31	<code>DemoData.Seed(context);</code> → Carga el dataset determinista de AceriaData.
Línea 32	<code>context.ChangeTracker.Clear();</code> → Limpia tracking antes de la demostración.
Línea 33	<code>` `</code> → Separa bloques lógicos.
Línea 34	<code>var useCase = scope.ServiceProvider.GetRequiredService<IdentityResolutionUseCase>();</code> → Resuelve una dependencia obligatoria desde el ámbito.
Línea 35	<code>useCase.Ejecutar();</code> → Participa directamente en el flujo validado del checkpoint: <code>useCase.Ejecutar();</code>
Línea 36	<code>` `</code> → Separa bloques lógicos.
Línea 37	<code>Console.WriteLine("4.3 OK");</code> → Publica evidencia observable en la consola.

Paso 7: Compilar

```
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

El build debe finalizar sin errores; el delta se propaga a los estados posteriores.

Paso 8: Ejecutar en LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

POWERSHELL

La salida debe terminar con 4.3 OK. Las aserciones internas fallan si la evidencia no coincide.

Paso 9: Diagnóstico técnico

Reto resuelto. Predecir cuántas instancias habrá cuando cuatro relaciones apunten a dos aleaciones distintas.

No uses solo tiempo: revisa SQL/roundtrips, cardinalidad, columnas, tracking y frontera de materialización.

Paso 10: Cierre acumulativo

1. Confirma el marcador E2E.
2. Comprueba que EnsureCreated no aparece.
3. Conserva las migraciones heredadas.
4. Verifica que Application no depende de EF Core.
5. Compara con el checkpoint anterior y documenta el delta.

Errores comunes revisados

- Confundir una medición aislada con una conclusión de rendimiento.
- Materializar antes de terminar filtros o proyecciones sin intención.
- Aplicar una técnica por regla general en lugar de observar la consulta.
- Relajar una aserción para ocultar un fallo en vez de corregir su causa.

Trazabilidad con la práctica fuente

La práctica fuente de 4.3 incluía además los siguientes focos docentes:

- AsNoTracking frente a AsNoTrackingWithIdentityResolution.
- Conteo por referencia usando ReferenceEqualityComparer.
- Escenario donde una misma clave aparece varias veces en el resultado.

Tratamiento en el M4 definitivo. La fuente usaba planchas compartidas, pero PlanchaAcero pertenece a una sola orden. La práctica definitiva usa Aleacion, que sí es una entidad compartida por varias relaciones y permite demostrar identidad duplicada de forma real.

Reto de ampliación procedente de la fuente. Compara por referencia las instancias de una aleación compartida con y sin Identity Resolution.

Errores de la fuente que deben seguir siendo diagnosticables

- No usar una entidad que nunca puede repetirse para demostrar resolución de identidad.
- No confundir igualdad de clave con igualdad de referencia.
- No dejar tracking previo activo durante la comparación.

Analogía operativa

La resolución de identidad evita crear dos fichas físicas para la misma clave dentro de una consulta.

Resultado esperado

El checkpoint 4.3 queda ejecutable, trazado y reproducible, y sirve como baseline física del punto siguiente.

Conexión con el siguiente punto

El siguiente estado es 4.4 y parte físicamente de este checkpoint.

Punto 4.4 — Problema N+1: identificación y causas

Contexto del proyecto

Este checkpoint continúa M04/PROYECTO/4.3. Conserva solución, capas, filtros, índices y migraciones; M4 no crea migraciones vacías.

Objetivo práctico

La baseline 3.12 tiene Lazy Loading desactivado. El N+1 se provoca de forma explícita: una consulta para órdenes y una adicional por orden. Un interceptor cuenta DbCommand reales.

Paso 1: Abrir el checkpoint

```
cd M04/PROYECTO/4.4
dotnet restore AceriaData.sln
```

POWERSHELL

Paso 2: Comprobar migraciones

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

POWERSHELL

Debe seguir apareciendo M2_2_12_Architecture.

Paso 3: Identificar el delta docente

El archivo principal del delta es src/AceriaData.Infrastructure/Repositories/Rendimiento44.cs. El checkpoint conserva todo el estado anterior.

Paso 4: Implementar y estudiar Infrastructure

```
using AceriaData.Application.Dtos;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public sealed partial class OrdenRepositorio
{
    public NMasUnoMetricaDto EjecutarNMasUnoM4()
    {
        SqlCommandCounterInterceptor.Instance.Reset();

        var ordenes = _context.OrdenesFabricacion
            .AsNoTracking()
            .OrderBy(o => o.Id)
            .Select(o => new { o.Id, o.NumeroOrden })
            .ToList();

        var totalPlanchas = 0;
        foreach (var orden in ordenes)
        {
            totalPlanchas += _context.PlanchasAcero
                .AsNoTracking()
                .Count(p => p.OrdenId == orden.Id);
        }

        return new NMasUnoMetricaDto
        {
            Ordenes = ordenes.Count,
            ConsultasSql = checked((int)SqlCommandCounterInterceptor.Instance.Count),
            Planchas = totalPlanchas
        };
    }
}
```

CSHARP

Archivo complementario: src/AceriaData.Infrastructure/SqlCommandCounterInterceptor.cs

CSHARP

```

using System.Data.Common;
using Microsoft.EntityFrameworkCore.Diagnostics;

namespace AceriaData.Infrastructure;

public sealed class SqlCommandCounterInterceptor : DbCommandInterceptor
{
    public static SqlCommandCounterInterceptor Instance { get; } = new();

    private long _count;
    public long Count => Interlocked.Read(ref _count);

    public void Reset() => Interlocked.Exchange(ref _count, 0);

    private void Increment() => Interlocked.Increment(ref _count);

    public override InterceptionResult<DbDataReader> ReaderExecuting(
        DbCommand command,
        CommandEventData eventData,
        InterceptionResult<DbDataReader> result)
    {
        Increment();
        return result;
    }

    public override InterceptionResult<object> ScalarExecuting(
        DbCommand command,
        CommandEventData eventData,
        InterceptionResult<object> result)
    {
        Increment();
        return result;
    }

    public override InterceptionResult<int> NonQueryExecuting(
        DbCommand command,
        CommandEventData eventData,
        InterceptionResult<int> result)
    {
        Increment();
        return result;
    }
}

```

Application no recibe DbContext, IQueryable ni referencias a Microsoft.EntityFrameworkCore.

Paso 5: Implementar el caso de uso

CSHARP

```

using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class NMasUnoUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public NMasUnoUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== 4.4 PROBLEMA N+1 ===");
        var metrica = _unidad.Ordenes.EjecutarNMasUnoM4();

        if (metrica.ConsultasSql != metrica.Ordenes + 1)
            throw new InvalidOperationException(
                $"4.4: se esperaban N+1 consultas; obtenidas {metrica.ConsultasSql} para N={metrica.Ordenes}.");

        Console.WriteLine(
            $"Ordenes={metrica.Ordenes} | Planchas={metrica.Planchas} | Consultas SQL={metrica.ConsultasSql}");
        Console.WriteLine(
            "La demostracion genera N+1 de forma explicita; Lazy Loading permanece desactivado en la baseline
3.12.");
    }
}

```

Explicación línea a línea — NMasUnoUseCase.cs

Línea 1

using AceriaData.Application.Interfaces; → Importa tipos o extensiones requeridos por esta implementación.

Línea 2	<code>` `</code> → Separa bloques lógicos.
Línea 3	<code>namespace AceriaData.Application.UseCases;</code> → Sitúa el archivo en la capa y espacio de nombres correspondiente.
Línea 4	<code>` `</code> → Separa bloques lógicos.
Línea 5	<code>public sealed class NMasUnoUseCase</code> → Declara la clase concreta usada por el checkpoint.
Línea 6	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 7	<code>private readonly IUnidadDeTrabajo _unidad;</code> → Declara la dependencia conservada por la instancia.
Línea 8	<code>public NMasUnoUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;</code> → Constructor del caso de uso e inyección de la unidad de trabajo.
Línea 9	<code>` `</code> → Separa bloques lógicos.
Línea 10	<code>public void Ejecutar()</code> → Define el flujo principal validado por el E2E.
Línea 11	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 12	<code>Console.WriteLine("=== 4.4 PROBLEMA N+1 ===");</code> → Publica evidencia observable en la consola.
Línea 13	<code>var metrica = _unidad.Ordenes.EjecutarNMasUnoM4();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 14	<code>` `</code> → Separa bloques lógicos.
Línea 15	<code>if (metrica.ConsultasSql != metrica.Ordenes + 1)</code> → Comprueba una condición contractual del E2E.
Línea 16	<code>throw new InvalidOperationException(</code> → Hace fallar el checkpoint si la evidencia no coincide.
Línea 17	<code> \$"4.4: se esperaban N+1 consultas; obtenidas {metrica.ConsultasSql} para N={metrica.Ordenes}.");</code> → Participa directamente en el flujo validado del checkpoint: <code> \$"4.4: se esperaban N+1 consultas; obtenidas {metrica.ConsultasSql} para N={metrica.Ordenes}.");</code>
Línea 18	<code>` `</code> → Separa bloques lógicos.
Línea 19	<code>Console.WriteLine(</code> → Publica evidencia observable en la consola.
Línea 20	<code> \$"Ordenes={metrica.Ordenes} Planchas={metrica.Planchas} Consultas SQL={metrica.ConsultasSql}");</code> → Participa directamente en el flujo validado del checkpoint: <code> \$"Ordenes={metrica.Ordenes} Planchas={metrica.Planchas} Consultas SQL={metrica.ConsultasSql}");</code>
Línea 21	<code>Console.WriteLine(</code> → Publica evidencia observable en la consola.
Línea 22	<code>"La demostracion genera N+1 de forma explicita; Lazy Loading permanece desactivado en la baseline 3.12.");</code> → Participa directamente en el flujo validado del checkpoint: <code>"La demostracion genera N+1 de forma explicita; Lazy Loading permanece desactivado en la baseline 3.12.");</code>
Línea 23	<code>}</code> → Delimita el bloque sintáctico asociado.
Línea 24	<code>}</code> → Delimita el bloque sintáctico asociado.

Paso 6: Preparar el composition root

CSHARP

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<BuenasPracticasUseCase>();
services.AddScoped<AnalisisSqlUseCase>();
services.AddScoped<TrackingUseCase>();
services.AddScoped<IdentityResolutionUseCase>();
services.AddScoped<NMasUnoUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true,
    ValidateScopes = true });
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<NMasUnoUseCase>();
useCase.Ejecutar();

Console.WriteLine("4.4 OK");
```

Explicación línea a línea — Program.cs

Línea 1	using AceriaData.Application.UseCases; → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	using AceriaData.ConsoleApp; → Importa tipos o extensiones requeridos por esta implementación.
Línea 3	using AceriaData.Infrastructure; → Importa tipos o extensiones requeridos por esta implementación.
Línea 4	using AceriaData.Infrastructure.Persistence; → Importa tipos o extensiones requeridos por esta implementación.
Línea 5	using Microsoft.EntityFrameworkCore; → Importa tipos o extensiones requeridos por esta implementación.
Línea 6	using Microsoft.Extensions.Configuration; → Importa tipos o extensiones requeridos por esta implementación.
Línea 7	using Microsoft.Extensions.DependencyInjection; → Importa tipos o extensiones requeridos por esta implementación.
Línea 8	`` → Separa bloques lógicos.
Línea 9	var configuration = new ConfigurationBuilder() → Calcula y conserva el resultado que será validado o mostrado.
Línea 10	.SetBasePath(AppContext.BaseDirectory) → Participa directamente en el flujo validado del checkpoint: .SetBasePath(AppContext.BaseDirectory)

Línea 11	<code>.AddJsonFile("appsettings.json", optional: false) → Participa directamente en el flujo validado del checkpoint: .AddJsonFile("appsettings.json", optional: false)</code>
Línea 12	<code>.AddEnvironmentVariables() → Participa directamente en el flujo validado del checkpoint: .AddEnvironmentVariables()</code>
Línea 13	<code>.Build(); → Participa directamente en el flujo validado del checkpoint: .Build();</code>
Línea 14	<code>` ` → Separa bloques lógicos.</code>
Línea 15	<code>var connectionString = configuration.GetConnectionString("AceriaDB") → Calcula y conserva el resultado que será validado o mostrado.</code>
Línea 16	<code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB"); → Participa directamente en el flujo validado del checkpoint: ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");</code>
Línea 17	<code>` ` → Separa bloques lógicos.</code>
Línea 18	<code>var services = new ServiceCollection(); → Calcula y conserva el resultado que será validado o mostrado.</code>
Línea 19	<code>services.AddAceriaInfrastructure(connectionString); → Participa directamente en el flujo validado del checkpoint: services.AddAceriaInfrastructure(connectionString);</code>
Línea 20	<code>services.AddScoped<BuenasPracticasUseCase>(); → Registra el caso de uso con ciclo de vida Scoped.</code>
Línea 21	<code>services.AddScoped<AnalisisSqlUseCase>(); → Registra el caso de uso con ciclo de vida Scoped.</code>
Línea 22	<code>services.AddScoped<TrackingUseCase>(); → Registra el caso de uso con ciclo de vida Scoped.</code>
Línea 23	<code>services.AddScoped<IdentityResolutionUseCase>(); → Registra el caso de uso con ciclo de vida Scoped.</code>
Línea 24	<code>services.AddScoped<NMasUnoUseCase>(); → Registra el caso de uso con ciclo de vida Scoped.</code>
Línea 25	<code>` ` → Separa bloques lógicos.</code>
Línea 26	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true, ValidateScopes = true }); → Importa tipos o extensiones requeridos por esta implementación.</code>
Línea 27	<code>using var scope = provider.CreateScope(); → Importa tipos o extensiones requeridos por esta implementación.</code>
Línea 28	<code>` ` → Separa bloques lógicos.</code>
Línea 29	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>(); → Resuelve una dependencia obligatoria desde el ámbito.</code>
Línea 30	<code>context.Database.EnsureDeleted(); → Reinicia la base de demostración para un E2E determinista.</code>
Línea 31	<code>context.Database.Migrate(); → Aplica la historia real de migraciones heredada.</code>
Línea 32	<code>DemoData.Seed(context); → Carga el dataset determinista de AceriaData.</code>
Línea 33	<code>context.ChangeTracker.Clear(); → Limpia tracking antes de la demostración.</code>
Línea 34	<code>` ` → Separa bloques lógicos.</code>

Línea 35	<code>var useCase = scope.ServiceProvider.GetRequiredService<NMasUnoUseCase>();</code> → Resuelve una dependencia obligatoria desde el ámbito.
Línea 36	<code>useCase.Ejecutar();</code> → Participa directamente en el flujo validado del checkpoint: <code>useCase.Ejecutar();</code>
Línea 37	<code>` `</code> → Separa bloques lógicos.
Línea 38	<code>Console.WriteLine("4.4 OK");</code> → Publica evidencia observable en la consola.

Paso 7: Compilar

```
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

El build debe finalizar sin errores; el delta se propaga a los estados posteriores.

Paso 8: Ejecutar en LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

POWERSHELL

La salida debe terminar con 4.4 OK. Las aserciones internas fallan si la evidencia no coincide.

Paso 9: Diagnóstico técnico

Reto resuelto. Calcular y después medir cuántos comandos se producen para N órdenes.

No uses solo tiempo: revisa SQL/roundtrips, cardinalidad, columnas, tracking y frontera de materialización.

Paso 10: Cierre acumulativo

1. Confirma el marcador E2E.
2. Comprueba que EnsureCreated no aparece.
3. Conserva las migraciones heredadas.
4. Verifica que Application no depende de EF Core.
5. Compara con el checkpoint anterior y documenta el delta.

Errores comunes revisados

- Confundir una medición aislada con una conclusión de rendimiento.
- Materializar antes de terminar filtros o proyecciones sin intención.
- Aplicar una técnica por regla general en lugar de observar la consulta.
- Relajar una aserción para ocultar un fallo en vez de corregir su causa.

Trazabilidad con la práctica fuente

La práctica fuente de 4.4 incluía además los siguientes focos docentes:

- Identificación de N+1, sus causas y relación con navegaciones.
- Conteo real de comandos SQL y comparación con una alternativa sin N+1.
- Variantes conceptuales con Lazy Loading, consultas en bucle, FirstOrDefault y proyecciones.

Tratamiento en el M4 definitivo. Lazy Loading permanece desactivado en la baseline. Por eso el N+1 se provoca explícitamente mediante una consulta por orden y se mide con DbCommandInterceptor, sin depender de comportamiento oculto.

Reto de ampliación procedente de la fuente. Provoca N+1 al consultar detalle por orden y compáralo conceptualmente con una carga anticipada o proyección.

Errores de la fuente que deben seguir siendo diagnosticables

- No asumir que acceder a una navegación ejecutará SQL cuando Lazy Loading está desactivado.

- No inferir N+1 por intuición: contar comandos reales.
- No mezclar estado previo del contexto en la medición.

Analogía operativa

N+1 es pedir una lista y volver a la ventanilla una vez por cada elemento.

Resultado esperado

El checkpoint 4.4 queda ejecutable, trazado y reproducible, y sirve como baseline física del punto siguiente.

Conexión con el siguiente punto

El siguiente estado es 4.5 y parte físicamente de este checkpoint.

Punto 4.5 — Solución a N+1: Include, proyecciones y Split Queries

Contexto del proyecto

Este checkpoint continúa M04/PROYECTO/4.4. Conserva solución, capas, filtros, índices y migraciones; M4 no crea migraciones vacías.

Objetivo práctico

No existe una solución universal al N+1. Include sirve para grafos; una proyección cuando solo se necesitan campos concretos; SplitQuery puede reducir explosión cartesiana con varias colecciones a costa de más roundtrips.

Paso 1: Abrir el checkpoint

```
cd M04/PROYECTO/4.5
dotnet restore AceriaData.sln
```

POWERSHELL

Paso 2: Comprobar migraciones

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

POWERSHELL

Debe seguir apareciendo M2_2_12_Architecture.

Paso 3: Identificar el delta docente

El archivo principal del delta es src/AceriaData.Infrastructure/Repositories/Rendimiento45.cs. El checkpoint conserva todo el estado anterior.

Paso 4: Implementar y estudiar Infrastructure

```
using AceriaData.Application.Dtos;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public sealed partial class OrdenRepositorio
{
    public SolucionNMasUnoMetricaDto EjecutarIncludeContraNMasUnoM4()
    {
        SqlCommandCounterInterceptor.Instance.Reset();
        var ordenes = _context.OrdenesFabricacion
            .AsNoTracking()
            .Include(o => o.Planchas)
            .OrderBy(o => o.Id)
    }
}
```

CSHARP

```

        .ToList();

        return new SolucionNMasUnoMetricaDto
        {
            Ordenes = ordenes.Count,
            ConsultasSql = checked((int)SqlCommandCounterInterceptor.Instance.Count),
            ElementosRelacionados = ordenes.Sum(o => o.Planchas.Count)
        };
    }

    public SolucionNMasUnoMetricaDto EjecutarProyeccionContraNMasUnoM4()
    {
        SqlCommandCounterInterceptor.Instance.Reset();
        var ordenes = _context.OrdenesFabricacion
            .AsNoTracking()
            .OrderBy(o => o.Id)
            .Select(o => new { o.Id, TotalPlanchas = o.Planchas.Count })
            .ToList();

        return new SolucionNMasUnoMetricaDto
        {
            Ordenes = ordenes.Count,
            ConsultasSql = checked((int)SqlCommandCounterInterceptor.Instance.Count),
            ElementosRelacionados = ordenes.Sum(o => o.TotalPlanchas)
        };
    }

    public SolucionNMasUnoMetricaDto EjecutarSplitQueryContraNMasUnoM4()
    {
        SqlCommandCounterInterceptor.Instance.Reset();
        var ordenes = _context.OrdenesFabricacion
            .AsNoTrackingWithIdentityResolution()
            .Include(o => o.Planchas)
            .Include(o => o.OrdenesAleaciones)
            .ThenInclude(oa => oa.Aleacion)
            .AsSplitQuery()
            .OrderBy(o => o.Id)
            .ToList();

        return new SolucionNMasUnoMetricaDto
        {
            Ordenes = ordenes.Count,
            ConsultasSql = checked((int)SqlCommandCounterInterceptor.Instance.Count),
            ElementosRelacionados = ordenes.Sum(o => o.Planchas.Count + o.OrdenesAleaciones.Count)
        };
    }
}

```

Application no recibe DbContext, IQueryable ni referencias a Microsoft.EntityFrameworkCore.

Paso 5: Implementar el caso de uso

CSHARP

```

using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class SolucionesNMasUnoUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public SolucionesNMasUnoUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== 4.5 SOLUCIONES AL N+1 ===");
        var include = _unidad.Ordenes.EjecutarIncludeContraNMasUnoM4();
        var proyeccion = _unidad.Ordenes.EjecutarProyeccionContraNMasUnoM4();
        var split = _unidad.Ordenes.EjecutarSplitQueryContraNMasUnoM4();

        if (include.ConsultasSql != 1 || proyeccion.ConsultasSql != 1)
            throw new InvalidOperationException("4.5: Include/proyeccion no redujeron la carga a una consulta.");
        if (split.ConsultasSql != 3)
            throw new InvalidOperationException(
                $"4.5: se esperaban 3 consultas en SplitQuery con dos colecciones; obtenidas {split.ConsultasSql}.");

        Console.WriteLine($"Include: {include.ConsultasSql} consulta.");
        Console.WriteLine($"Proyeccion: {proyeccion.ConsultasSql} consulta.");
        Console.WriteLine($"SplitQuery: {split.ConsultasSql} consultas para evitar explosion cartesiana con dos colecciones.");
    }
}

```

Explicación línea a línea — SolucionesNMasUnoUseCase.cs

Línea 1	<code>using AceriaData.Application.Interfaces;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	<code>` `</code> → Separa bloques lógicos.
Línea 3	<code>namespace AceriaData.Application.UseCases;</code> → Sitúa el archivo en la capa y espacio de nombres correspondiente.
Línea 4	<code>` `</code> → Separa bloques lógicos.
Línea 5	<code>public sealed class SolucionesNMasUnoUseCase</code> → Declara la clase concreta usada por el checkpoint.
Línea 6	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 7	<code>private readonly IUnidadDeTrabajo _unidad;</code> → Declara la dependencia conservada por la instancia.
Línea 8	<code>public SolucionesNMasUnoUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;</code> → Constructor del caso de uso e inyección de la unidad de trabajo.
Línea 9	<code>` `</code> → Separa bloques lógicos.
Línea 10	<code>public void Ejecutar()</code> → Define el flujo principal validado por el E2E.
Línea 11	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 12	<code>Console.WriteLine("=== 4.5 SOLUCIONES AL N+1 ===");</code> → Publica evidencia observable en la consola.
Línea 13	<code>var include = _unidad.Ordenes.EjecutarIncludeContraNMasUnoM4();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 14	<code>var proyeccion = _unidad.Ordenes.EjecutarProyeccionContraNMasUnoM4();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 15	<code>var split = _unidad.Ordenes.EjecutarSplitQueryContraNMasUnoM4();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 16	<code>` `</code> → Separa bloques lógicos.
Línea 17	<code>if (include.ConsultasSql != 1 proyeccion.ConsultasSql != 1)</code> → Comprueba una condición contractual del E2E.
Línea 18	<code>throw new InvalidOperationException("4.5: Include/proyeccion no redujeron la carga a una consulta.");</code> → Hace fallar el checkpoint si la evidencia no coincide.
Línea 19	<code>if (split.ConsultasSql != 3)</code> → Comprueba una condición contractual del E2E.
Línea 20	<code>throw new InvalidOperationException(</code> → Hace fallar el checkpoint si la evidencia no coincide.
Línea 21	<code>\$"4.5: se esperaban 3 consultas en SplitQuery con dos colecciones; obtenidas {split.ConsultasSql}.");</code> → Participa directamente en el flujo validado del checkpoint: <code>\$"4.5: se esperaban 3 consultas en SplitQuery con dos colecciones; obtenidas {split.ConsultasSql}."</code> ;
Línea 22	<code>` `</code> → Separa bloques lógicos.
Línea 23	<code>Console.WriteLine(\$"Include: {include.ConsultasSql} consulta.");</code> → Publica evidencia observable en la consola.

Línea 24	Console.WriteLine(\$"Proyeccion: {proyeccion.ConsultasSql} consulta."); → Publica evidencia observable en la consola.
Línea 25	Console.WriteLine(\$"SplitQuery: {split.ConsultasSql} consultas para evitar explosion cartesiana con dos colecciones."); → Publica evidencia observable en la consola.
Línea 26	} → Delimita el bloque sintáctico asociado.
Línea 27	} → Delimita el bloque sintáctico asociado.

Paso 6: Preparar el composition root

CSHARP

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<BuenasPracticasUseCase>();
services.AddScoped<AnalisisSqlUseCase>();
services.AddScoped<TrackingUseCase>();
services.AddScoped<IdentityResolutionUseCase>();
services.AddScoped<NMasUnoUseCase>();
services.AddScoped<SolucionesNMasUnoUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true,
    ValidateScopes = true });
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<SolucionesNMasUnoUseCase>();
useCase.Ejecutar();

Console.WriteLine("4.5 OK");
```

Explicación línea a línea — Program.cs

Línea 1	using AceriaData.Application.UseCases; → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	using AceriaData.ConsoleApp; → Importa tipos o extensiones requeridos por esta implementación.
Línea 3	using AceriaData.Infrastructure; → Importa tipos o extensiones requeridos por esta implementación.
Línea 4	using AceriaData.Infrastructure.Persistence; → Importa tipos o extensiones requeridos por esta implementación.
Línea 5	using Microsoft.EntityFrameworkCore; → Importa tipos o extensiones requeridos por esta implementación.
Línea 6	using Microsoft.Extensions.Configuration; → Importa tipos o extensiones requeridos por esta implementación.

Línea 7	<code>using Microsoft.Extensions.DependencyInjection;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 8	<code>` `</code> → Separa bloques lógicos.
Línea 9	<code>var configuration = new ConfigurationBuilder();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 10	<code>.SetBasePath(AppContext.BaseDirectory)</code> → Participa directamente en el flujo validado del checkpoint: <code>.SetBasePath(AppContext.BaseDirectory)</code>
Línea 11	<code>.AddJsonFile("appsettings.json", optional: false)</code> → Participa directamente en el flujo validado del checkpoint: <code>.AddJsonFile("appsettings.json", optional: false)</code>
Línea 12	<code>.AddEnvironmentVariables();</code> → Participa directamente en el flujo validado del checkpoint: <code>.AddEnvironmentVariables();</code>
Línea 13	<code>.Build();</code> → Participa directamente en el flujo validado del checkpoint: <code>.Build();</code>
Línea 14	<code>` `</code> → Separa bloques lógicos.
Línea 15	<code>var connectionString = configuration.GetConnectionString("AceriaDB");</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 16	<code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");</code> → Participa directamente en el flujo validado del checkpoint: <code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");</code>
Línea 17	<code>` `</code> → Separa bloques lógicos.
Línea 18	<code>var services = new ServiceCollection();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 19	<code>services.AddAceriaInfrastructure(connectionString);</code> → Participa directamente en el flujo validado del checkpoint: <code>services.AddAceriaInfrastructure(connectionString);</code>
Línea 20	<code>services.AddScoped<BuenasPracticasUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 21	<code>services.AddScoped<AnalisisSqlUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 22	<code>services.AddScoped<TrackingUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 23	<code>services.AddScoped<IdentityResolutionUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 24	<code>services.AddScoped<NMasUnoUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 25	<code>services.AddScoped<SolucionesNMasUnoUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 26	<code>` `</code> → Separa bloques lógicos.
Línea 27	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true, ValidateScopes = true });</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 28	<code>using var scope = provider.CreateScope();</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 29	<code>` `</code> → Separa bloques lógicos.

Línea 30	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve una dependencia obligatoria desde el ámbito.
Línea 31	<code>context.Database.EnsureDeleted();</code> → Reinicia la base de demostración para un E2E determinista.
Línea 32	<code>context.Database.Migrate();</code> → Aplica la historia real de migraciones heredada.
Línea 33	<code>DemoData.Seed(context);</code> → Carga el dataset determinista de <code>AceriaData</code> .
Línea 34	<code>context.ChangeTracker.Clear();</code> → Limpia tracking antes de la demostración.
Línea 35	<code>``</code> → Separa bloques lógicos.
Línea 36	<code>var useCase = scope.ServiceProvider.GetRequiredService<SolucionesNMasUnoUseCase>();</code> → Resuelve una dependencia obligatoria desde el ámbito.
Línea 37	<code>useCase.Ejecutar();</code> → Participa directamente en el flujo validado del checkpoint: <code>useCase.Ejecutar();</code>
Línea 38	<code>``</code> → Separa bloques lógicos.
Línea 39	<code>Console.WriteLine("4.5 OK");</code> → Publica evidencia observable en la consola.

Paso 7: Compilar

```
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

El build debe finalizar sin errores; el delta se propaga a los estados posteriores.

Paso 8: Ejecutar en LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

POWERSHELL

La salida debe terminar con 4.5 OK. Las aserciones internas fallan si la evidencia no coincide.

Paso 9: Diagnóstico técnico

Reto resuelto. Elegir entre Include, proyección o SplitQuery para tres escenarios y justificar el coste dominante.

No uses solo tiempo: revisa SQL/roundtrips, cardinalidad, columnas, tracking y frontera de materialización.

Paso 10: Cierre acumulativo

1. Confirma el marcador E2E.
2. Comprueba que `EnsureCreated` no aparece.
3. Conserva las migraciones heredadas.
4. Verifica que `Application` no depende de EF Core.
5. Compara con el checkpoint anterior y documenta el delta.

Errores comunes revisados

- Confundir una medición aislada con una conclusión de rendimiento.
- Materializar antes de terminar filtros o proyecciones sin intención.
- Aplicar una técnica por regla general en lugar de observar la consulta.
- Relajar una aserción para ocultar un fallo en vez de corregir su causa.

Trazabilidad con la práctica fuente

La práctica fuente de 4.5 incluía además los siguientes focos docentes:

- Include y ThenInclude para cargar grafos.
- Proyecciones para obtener solo los datos necesarios.
- AsSplitQuery como alternativa cuando existen varias colecciones.

Tratamiento en el M4 definitivo. El checkpoint compara alternativas contando comandos reales. SplitQuery no se presenta como regla universal: se usa en un grafo con dos colecciones donde el trade-off es observable.

Reto de ampliación procedente de la fuente. Combina Include, ThenInclude, Identity Resolution y SplitQuery en un grafo con planchas y aleaciones y justifica el número de comandos.

Errores de la fuente que deben seguir siendo diagnosticables

- No aplicar SplitQuery por defecto sin observar la forma del grafo.
- No comparar tiempos sin aislar tracking y dataset.
- No confundir evitar N+1 con garantizar una única consulta.

Analogía operativa

Optimizar N+1 es decidir si conviene traer el expediente completo, un resumen o varios lotes coordinados.

Resultado esperado

El checkpoint 4.5 queda ejecutable, trazado y reproducible, y sirve como baseline física del punto siguiente.

Conexión con el siguiente punto

El siguiente estado es 4.6 y parte físicamente de este checkpoint.

Punto 4.6 — Over-fetching: causas y soluciones

Contexto del proyecto

Este checkpoint continúa M04/PROYECTO/4.5. Conserva solución, capas, filtros, índices y migraciones; M4 no crea migraciones vacías.

Objetivo práctico

Over-fetching se diagnostica observando la forma real del SELECT. La práctica compara igual cardinalidad con entidad completa frente a proyección DTO.

Paso 1: Abrir el checkpoint

```
cd M04/PROYECTO/4.6
dotnet restore AceriaData.sln
```

POWERSHELL

Paso 2: Comprobar migraciones

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

POWERSHELL

Debe seguir apareciendo M2_2_12_Architecture.

Paso 3: Identificar el delta docente

El archivo principal del delta es src/AceriaData.Infrastructure/Repositories/Rendimiento46.cs. El checkpoint conserva todo el estado anterior.

Paso 4: Implementar y estudiar Infrastructure

CSHARP

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public sealed partial class OrdenRepositorio
{
    private IQueryable<OrdenFabricacion> PendientesM4() => _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Estado == "Pendiente")
        .OrderBy(o => o.FechaCreacion)
        .ThenBy(o => o.Id);

    public List<OrdenFabricacion> ObtenerPendientesEntidadCompletaM4() =>
        PendientesM4().ToList();

    public List<OrdenResumenDto> ObtenerPendientesProyectadasM4() =>
        PendientesM4()
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();

    public string ObtenerSqlPendientesEntidadCompletaM4() =>
        PendientesM4().ToQueryString();

    public string ObtenerSqlPendientesProyectadasM4() =>
        PendientesM4()
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToQueryString();
}
```

Application no recibe DbContext, IQueryable ni referencias a Microsoft.EntityFrameworkCore.

Paso 5: Implementar el caso de uso

CSHARP

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class OverFetchingUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public OverFetchingUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== 4.6 OVER-FETCHING ===");
        var completas = _unidad.Ordenes.ObtenerPendientesEntidadCompletaM4();
        var proyectadas = _unidad.Ordenes.ObtenerPendientesProyectadasM4();
        var sqlCompleto = _unidad.Ordenes.ObtenerSqlPendientesEntidadCompletaM4();
        var sqlProyectado = _unidad.Ordenes.ObtenerSqlPendientesProyectadasM4();

        if (completas.Count != proyectadas.Count)
            throw new InvalidOperationException("4.6: la proyeccion cambio la cardinalidad.");
        if (!sqlCompleto.Contains("Observaciones", StringComparison.OrdinalIgnoreCase))
            throw new InvalidOperationException("4.6: el SQL completo no evidencia columnas extra.");
        if (sqlProyectado.Contains("Observaciones", StringComparison.OrdinalIgnoreCase))
            throw new InvalidOperationException("4.6: la proyeccion sigue recuperando Observaciones.");

        Console.WriteLine($"Filas equivalentes: {completas.Count}");
        Console.WriteLine("--- SQL entidad completa ---");
        Console.WriteLine(sqlCompleto);
        Console.WriteLine("--- SQL proyeccion ---");
        Console.WriteLine(sqlProyectado);
    }
}
```

```
}
}
```

Explicación línea a línea — OverFetchingUseCase.cs

Línea 1	<code>using AceriaData.Application.Interfaces;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	<code>` `</code> → Separa bloques lógicos.
Línea 3	<code>namespace AceriaData.Application.UseCases;</code> → Sitúa el archivo en la capa y espacio de nombres correspondiente.
Línea 4	<code>` `</code> → Separa bloques lógicos.
Línea 5	<code>public sealed class OverFetchingUseCase</code> → Declara la clase concreta usada por el checkpoint.
Línea 6	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 7	<code>private readonly IUnidadDeTrabajo _unidad;</code> → Declara la dependencia conservada por la instancia.
Línea 8	<code>public OverFetchingUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;</code> → Constructor del caso de uso e inyección de la unidad de trabajo.
Línea 9	<code>` `</code> → Separa bloques lógicos.
Línea 10	<code>public void Ejecutar()</code> → Define el flujo principal validado por el E2E.
Línea 11	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 12	<code>Console.WriteLine("=== 4.6 OVER-FETCHING ===");</code> → Publica evidencia observable en la consola.
Línea 13	<code>var completas = _unidad.Ordenes.ObtenerPendientesEntidadCompletaM4();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 14	<code>var proyectadas = _unidad.Ordenes.ObtenerPendientesProyectadasM4();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 15	<code>var sqlCompleto = _unidad.Ordenes.ObtenerSqlPendientesEntidadCompletaM4();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 16	<code>var sqlProyectado = _unidad.Ordenes.ObtenerSqlPendientesProyectadasM4();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 17	<code>` `</code> → Separa bloques lógicos.
Línea 18	<code>if (completas.Count != proyectadas.Count)</code> → Comprueba una condición contractual del E2E.
Línea 19	<code>throw new InvalidOperationException("4.6: la proyeccion cambio la cardinalidad.");</code> → Hace fallar el checkpoint si la evidencia no coincide.
Línea 20	<code>if (!sqlCompleto.Contains("Observaciones", StringComparison.OrdinalIgnoreCase))</code> → Comprueba una condición contractual del E2E.
Línea 21	<code>throw new InvalidOperationException("4.6: el SQL completo no evidencia columnas extra.");</code> → Hace fallar el checkpoint si la evidencia no coincide.
Línea 22	<code>if (sqlProyectado.Contains("Observaciones", StringComparison.OrdinalIgnoreCase))</code> → Comprueba una condición contractual del E2E.
Línea 23	<code>throw new InvalidOperationException("4.6: la proyeccion sigue recuperando Observaciones.");</code> → Hace fallar el checkpoint si la evidencia no coincide.

Línea 24	<code>` `</code> → Separa bloques lógicos.
Línea 25	<code>Console.WriteLine(\$"Filas equivalentes: {completas.Count}");</code> → Publica evidencia observable en la consola.
Línea 26	<code>Console.WriteLine("--- SQL entidad completa ---");</code> → Publica evidencia observable en la consola.
Línea 27	<code>Console.WriteLine(sqlCompleto);</code> → Publica evidencia observable en la consola.
Línea 28	<code>Console.WriteLine("--- SQL proyeccion ---");</code> → Publica evidencia observable en la consola.
Línea 29	<code>Console.WriteLine(sqlProyectado);</code> → Publica evidencia observable en la consola.
Línea 30	<code>}</code> → Delimita el bloque sintáctico asociado.
Línea 31	<code>}</code> → Delimita el bloque sintáctico asociado.

Paso 6: Preparar el composition root

CSHARP

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<BuenasPracticasUseCase>();
services.AddScoped<AnalisisSqlUseCase>();
services.AddScoped<TrackingUseCase>();
services.AddScoped<IdentityResolutionUseCase>();
services.AddScoped<NMasUnoUseCase>();
services.AddScoped<SolucionesNMasUnoUseCase>();
services.AddScoped<OverFetchingUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true,
ValidateScopes = true });
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<OverFetchingUseCase>();
useCase.Ejecutar();

Console.WriteLine("4.6 OK");
```

Explicación línea a línea — Program.cs

Línea 1	<code>using AceriaData.Application.UseCases;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	<code>using AceriaData.ConsoleApp;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 3	<code>using AceriaData.Infrastructure;</code> → Importa tipos o extensiones requeridos por esta implementación.

Línea 4	<code>using AceriaData.Infrastructure.Persistence;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 5	<code>using Microsoft.EntityFrameworkCore;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 6	<code>using Microsoft.Extensions.Configuration;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 7	<code>using Microsoft.Extensions.DependencyInjection;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 8	<code>` `</code> → Separa bloques lógicos.
Línea 9	<code>var configuration = new ConfigurationBuilder();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 10	<code>.SetBasePath(AppContext.BaseDirectory)</code> → Participa directamente en el flujo validado del checkpoint: <code>.SetBasePath(AppContext.BaseDirectory)</code>
Línea 11	<code>.AddJsonFile("appsettings.json", optional: false)</code> → Participa directamente en el flujo validado del checkpoint: <code>.AddJsonFile("appsettings.json", optional: false)</code>
Línea 12	<code>.AddEnvironmentVariables()</code> → Participa directamente en el flujo validado del checkpoint: <code>.AddEnvironmentVariables()</code>
Línea 13	<code>.Build();</code> → Participa directamente en el flujo validado del checkpoint: <code>.Build();</code>
Línea 14	<code>` `</code> → Separa bloques lógicos.
Línea 15	<code>var connectionString = configuration.GetConnectionString("AceriaDB");</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 16	<code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");</code> → Participa directamente en el flujo validado del checkpoint: <code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");</code>
Línea 17	<code>` `</code> → Separa bloques lógicos.
Línea 18	<code>var services = new ServiceCollection();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 19	<code>services.AddAceriaInfrastructure(connectionString);</code> → Participa directamente en el flujo validado del checkpoint: <code>services.AddAceriaInfrastructure(connectionString);</code>
Línea 20	<code>services.AddScoped<BuenasPracticasUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 21	<code>services.AddScoped<AnalisisSqlUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 22	<code>services.AddScoped<TrackingUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 23	<code>services.AddScoped<IdentityResolutionUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 24	<code>services.AddScoped<NMasUnoUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 25	<code>services.AddScoped<SolucionesNMasUnoUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 26	<code>services.AddScoped<OverFetchingUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 27	<code>` `</code> → Separa bloques lógicos.

Línea 28	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true, ValidateScopes = true });</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 29	<code>using var scope = provider.CreateScope();</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 30	<code>` `</code> → Separa bloques lógicos.
Línea 31	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve una dependencia obligatoria desde el ámbito.
Línea 32	<code>context.Database.EnsureDeleted();</code> → Reinicia la base de demostración para un E2E determinista.
Línea 33	<code>context.Database.Migrate();</code> → Aplica la historia real de migraciones heredada.
Línea 34	<code>DemoData.Seed(context);</code> → Carga el dataset determinista de <code>AceriaData</code> .
Línea 35	<code>context.ChangeTracker.Clear();</code> → Limpia tracking antes de la demostración.
Línea 36	<code>` `</code> → Separa bloques lógicos.
Línea 37	<code>var useCase = scope.ServiceProvider.GetRequiredService<OverFetchingUseCase>();</code> → Resuelve una dependencia obligatoria desde el ámbito.
Línea 38	<code>useCase.Ejecutar();</code> → Participa directamente en el flujo validado del checkpoint: <code>useCase.Ejecutar();</code>
Línea 39	<code>` `</code> → Separa bloques lógicos.
Línea 40	<code>Console.WriteLine("4.6 OK");</code> → Publica evidencia observable en la consola.

Paso 7: Compilar

```
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

El build debe finalizar sin errores; el delta se propaga a los estados posteriores.

Paso 8: Ejecutar en LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

POWERSHELL

La salida debe terminar con 4.6 OK. Las aserciones internas fallan si la evidencia no coincide.

Paso 9: Diagnóstico técnico

Reto resuelto. Identificar en el SQL qué columnas desaparecen al proyectar y relacionarlo con transferencia y materialización.

No uses solo tiempo: revisa SQL/roundtrips, cardinalidad, columnas, tracking y frontera de materialización.

Paso 10: Cierre acumulativo

1. Confirma el marcador E2E.
2. Comprueba que `EnsureCreated` no aparece.
3. Conserva las migraciones heredadas.
4. Verifica que `Application` no depende de EF Core.
5. Compara con el checkpoint anterior y documenta el delta.

Errores comunes revisados

- Confundir una medición aislada con una conclusión de rendimiento.
- Materializar antes de terminar filtros o proyecciones sin intención.
- Aplicar una técnica por regla general en lugar de observar la consulta.
- Relajar una aserción para ocultar un fallo en vez de corregir su causa.

Trazabilidad con la práctica fuente

La práctica fuente de 4.6 incluía además los siguientes focos docentes:

- Over-fetching de columnas y de filas.
- Proyecciones, filtros y paginación para reducir datos transferidos.
- Inspección del SQL para comparar entidad completa frente a shape reducido.

Tratamiento en el M4 definitivo. El checkpoint 4.6 demuestra directamente el over-fetching de columnas con SQL real. El over-fetching de filas y la paginación se mantienen en teoría y se ejecutan de forma específica en 4.10.

Reto de ampliación procedente de la fuente. Compara el SELECT de entidad completa y proyección y relaciona las columnas eliminadas con transferencia y materialización.

Errores de la fuente que deben seguir siendo diagnosticables

- No aplicar Skip sin un OrderBy determinista.
- No materializar antes de terminar filtros y proyecciones.
- No medir solo tiempo cuando el objetivo es demostrar volumen de datos.

Analogía operativa

Over-fetching es mover un palé entero cuando la siguiente estación solo necesita cuatro piezas.

Resultado esperado

El checkpoint 4.6 queda ejecutable, trazado y reproducible, y sirve como baseline física del punto siguiente.

Conexión con el siguiente punto

El siguiente estado es 4.7 y parte físicamente de este checkpoint.

Punto 4.7 — Consultas ineficientes: traducción y frontera cliente/servidor

Contexto del proyecto

Este checkpoint continúa M04/PROYECTO/4.6. Conserva solución, capas, filtros, índices y migraciones; M4 no crea migraciones vacías.

Objetivo práctico

En EF Core 8 un predicado no traducible dentro de Where no se evalúa silenciosamente en cliente: falla. La evaluación cliente exige una frontera explícita como IEnumerable. Las funciones sobre columnas pueden perjudicar sargabilidad y deben medirse.

Paso 1: Abrir el checkpoint

```
cd M04/PROYECTO/4.7
dotnet restore AceriaData.sln
```

POWERSHELL

Paso 2: Comprobar migraciones

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

POWERSHELL

Debe seguir apareciendo M2_2_12_Architecture.

Paso 3: Identificar el delta docente

El archivo principal del delta es src/AceriaData.Infrastructure/Repositories/Rendimiento47.cs. El checkpoint conserva todo el estado anterior.

Paso 4: Implementar y estudiar Infrastructure

```
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public sealed partial class OrdenRepository
{
    private static bool EstadoCoincideM4(string actual, string buscado) =>
        string.Equals(actual, buscado, StringComparison.OrdinalIgnoreCase);

    public bool FiltroPersonalizadoNoTraducibleFallaM4(string estado)
    {
        try
        {
            _ = _context.OrdenesFabricacion
                .AsNoTracking()
                .Where(o => EstadoCoincideM4(o.Estado, estado))
                .ToList();
            return false;
        }
        catch (InvalidOperationException)
        {
            return true;
        }
    }

    public int ContarConEvaluacionClienteExplicitaM4(string estado) =>
        _context.OrdenesFabricacion
            .AsNoTracking()
            .AsEnumerable()
            .Count(o => EstadoCoincideM4(o.Estado, estado));

    public string ObtenerSqlClienteConFuncionM4(string cliente)
    {
        var normalizado = cliente.ToLowerInvariant();
        return _context.OrdenesFabricacion
            .AsNoTracking()
            .Where(o => o.Cliente.ToLower() == normalizado)
            .ToString();
    }

    public string ObtenerSqlClienteDirectoM4(string cliente) =>
        _context.OrdenesFabricacion
            .AsNoTracking()
            .Where(o => o.Cliente == cliente)
            .ToString();
    }
}
```

CSHARP

Application no recibe DbContext, IQueryable ni referencias a Microsoft.EntityFrameworkCore.

Paso 5: Implementar el caso de uso

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class TraduccionConsultasUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public TraduccionConsultasUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
    }
}
```

CSHARP

```

Console.WriteLine("=== 4.7 TRADUCCION Y FRONTERA CLIENTE/SERVIDOR ===");

if (!_unidad.Ordenes.FiltroPersonalizadoNoTraducibleFallaM4("Pendiente"))
    throw new InvalidOperationException("4.7: EF Core no rechazo el filtro personalizado no traducible.");

var cliente = _unidad.Ordenes.ContarConEvaluacionClienteExplicitaM4("Pendiente");
if (cliente <= 0)
    throw new InvalidOperationException("4.7: la evaluacion cliente explicita no devolvio datos.");

var sqlFuncion = _unidad.Ordenes.ObtenerSqlClienteConFuncionM4("Constructora del Norte");
var sqlDirecto = _unidad.Ordenes.ObtenerSqlClienteDirectoM4("Constructora del Norte");

if (!sqlFuncion.Contains("LOWER", StringComparison.OrdinalIgnoreCase))
    throw new InvalidOperationException("4.7: no se observa LOWER en el SQL con funcion.");
if (sqlDirecto.Contains("LOWER", StringComparison.OrdinalIgnoreCase))
    throw new InvalidOperationException("4.7: la comparacion directa introdujo LOWER inesperadamente.");

Console.WriteLine("Filtro no traducible: InvalidOperationException observada.");
Console.WriteLine($"Evaluacion cliente explicita: {cliente} filas coincidentes.");
Console.WriteLine("--- SQL con funcion sobre columna ---");
Console.WriteLine(sqlFuncion);
Console.WriteLine("--- SQL con comparacion directa ---");
Console.WriteLine(sqlDirecto);
    }
}

```

Explicación línea a línea — TraduccionsConsultasUseCase.cs

Línea 1	using AceriaData.Application.Interfaces; → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	`` → Separa bloques lógicos.
Línea 3	namespace AceriaData.Application.UseCases; → Sitúa el archivo en la capa y espacio de nombres correspondiente.
Línea 4	`` → Separa bloques lógicos.
Línea 5	public sealed class TraduccionsConsultasUseCase → Declara la clase concreta usada por el checkpoint.
Línea 6	{ → Delimita el bloque sintáctico asociado.
Línea 7	private readonly IUnidadDeTrabajo _unidad; → Declara la dependencia conservada por la instancia.
Línea 8	public TraduccionsConsultasUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad; → Constructor del caso de uso e inyección de la unidad de trabajo.
Línea 9	`` → Separa bloques lógicos.
Línea 10	public void Ejecutar() → Define el flujo principal validado por el E2E.
Línea 11	{ → Delimita el bloque sintáctico asociado.
Línea 12	Console.WriteLine("=== 4.7 TRADUCCION Y FRONTERA CLIENTE/SERVIDOR ==="); → Publica evidencia observable en la consola.
Línea 13	`` → Separa bloques lógicos.
Línea 14	if (!_unidad.Ordenes.FiltroPersonalizadoNoTraducibleFallaM4("Pendiente")) → Comprueba una condición contractual del E2E.
Línea 15	throw new InvalidOperationException("4.7: EF Core no rechazo el filtro personalizado no traducible."); → Hace fallar el checkpoint si la evidencia no coincide.
Línea 16	`` → Separa bloques lógicos.

Línea 17	<code>var cliente = _unidad.Ordenes.ContarConEvaluacionClienteExplicitaM4("Pendiente");</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 18	<code>if (cliente <= 0)</code> → Comprueba una condición contractual del E2E.
Línea 19	<code>throw new InvalidOperationException("4.7: la evaluacion cliente explicita no devolvio datos.");</code> → Hace fallar el checkpoint si la evidencia no coincide.
Línea 20	<code>` `</code> → Separa bloques lógicos.
Línea 21	<code>var sqlFuncion = _unidad.Ordenes.ObtenerSqlClienteConFuncionM4("Constructora del Norte");</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 22	<code>var sqlDirecto = _unidad.Ordenes.ObtenerSqlClienteDirectoM4("Constructora del Norte");</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 23	<code>` `</code> → Separa bloques lógicos.
Línea 24	<code>if (!sqlFuncion.Contains("LOWER", StringComparison.OrdinalIgnoreCase))</code> → Comprueba una condición contractual del E2E.
Línea 25	<code>throw new InvalidOperationException("4.7: no se observa LOWER en el SQL con funcion.");</code> → Hace fallar el checkpoint si la evidencia no coincide.
Línea 26	<code>if (sqlDirecto.Contains("LOWER", StringComparison.OrdinalIgnoreCase))</code> → Comprueba una condición contractual del E2E.
Línea 27	<code>throw new InvalidOperationException("4.7: la comparacion directa introdujo LOWER inesperadamente.");</code> → Hace fallar el checkpoint si la evidencia no coincide.
Línea 28	<code>` `</code> → Separa bloques lógicos.
Línea 29	<code>Console.WriteLine("Filtro no traducible: InvalidOperationException observada.");</code> → Publica evidencia observable en la consola.
Línea 30	<code>Console.WriteLine(\$"Evaluacion cliente explicita: {cliente} filas coincidentes.");</code> → Publica evidencia observable en la consola.
Línea 31	<code>Console.WriteLine("--- SQL con funcion sobre columna ---");</code> → Publica evidencia observable en la consola.
Línea 32	<code>Console.WriteLine(sqlFuncion);</code> → Publica evidencia observable en la consola.
Línea 33	<code>Console.WriteLine("--- SQL con comparacion directa ---");</code> → Publica evidencia observable en la consola.
Línea 34	<code>Console.WriteLine(sqlDirecto);</code> → Publica evidencia observable en la consola.
Línea 35	<code>}</code> → Delimita el bloque sintáctico asociado.
Línea 36	<code>}</code> → Delimita el bloque sintáctico asociado.

Paso 6: Preparar el composition root

CSHARP

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
```

```

.SetBasePath(AppContext.BaseDirectory)
.AddJsonFile("appsettings.json", optional: false)
.AddEnvironmentVariables()
.Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<BuenasPracticasUseCase>();
services.AddScoped<AnalisisSqlUseCase>();
services.AddScoped<TrackingUseCase>();
services.AddScoped<IdentityResolutionUseCase>();
services.AddScoped<NMasUnoUseCase>();
services.AddScoped<SolucionesNMasUnoUseCase>();
services.AddScoped<OverFetchingUseCase>();
services.AddScoped<TraduccionConsultasUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true,
ValidateScopes = true });
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<TraduccionConsultasUseCase>();
useCase.Ejecutar();

Console.WriteLine("4.7 OK");

```

Explicación línea a línea — Program.cs

Línea 1	using AceriaData.Application.UseCases; → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	using AceriaData.ConsoleApp; → Importa tipos o extensiones requeridos por esta implementación.
Línea 3	using AceriaData.Infrastructure; → Importa tipos o extensiones requeridos por esta implementación.
Línea 4	using AceriaData.Infrastructure.Persistence; → Importa tipos o extensiones requeridos por esta implementación.
Línea 5	using Microsoft.EntityFrameworkCore; → Importa tipos o extensiones requeridos por esta implementación.
Línea 6	using Microsoft.Extensions.Configuration; → Importa tipos o extensiones requeridos por esta implementación.
Línea 7	using Microsoft.Extensions.DependencyInjection; → Importa tipos o extensiones requeridos por esta implementación.
Línea 8	`` → Separa bloques lógicos.
Línea 9	var configuration = new ConfigurationBuilder() → Calcula y conserva el resultado que será validado o mostrado.
Línea 10	.SetBasePath(AppContext.BaseDirectory) → Participa directamente en el flujo validado del checkpoint: .SetBasePath(AppContext.BaseDirectory)
Línea 11	.AddJsonFile("appsettings.json", optional: false) → Participa directamente en el flujo validado del checkpoint: .AddJsonFile("appsettings.json", optional: false)
Línea 12	.AddEnvironmentVariables() → Participa directamente en el flujo validado del checkpoint: .AddEnvironmentVariables()
Línea 13	.Build(); → Participa directamente en el flujo validado del checkpoint: .Build();

Línea 14	<code>` `</code> → Separa bloques lógicos.
Línea 15	<code>var connectionString = configuration.GetConnectionString("AceriaDB");</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 16	<code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");</code> → Participa directamente en el flujo validado del checkpoint: <code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");</code>
Línea 17	<code>` `</code> → Separa bloques lógicos.
Línea 18	<code>var services = new ServiceCollection();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 19	<code>services.AddAceriaInfrastructure(connectionString);</code> → Participa directamente en el flujo validado del checkpoint: <code>services.AddAceriaInfrastructure(connectionString);</code>
Línea 20	<code>services.AddScoped<BuenasPracticasUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 21	<code>services.AddScoped<AnalisisSqlUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 22	<code>services.AddScoped<TrackingUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 23	<code>services.AddScoped<IdentityResolutionUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 24	<code>services.AddScoped<NMasUnoUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 25	<code>services.AddScoped<SolucionesNMasUnoUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 26	<code>services.AddScoped<OverFetchingUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 27	<code>services.AddScoped<TraduccionConsultasUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 28	<code>` `</code> → Separa bloques lógicos.
Línea 29	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true, ValidateScopes = true });</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 30	<code>using var scope = provider.CreateScope();</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 31	<code>` `</code> → Separa bloques lógicos.
Línea 32	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve una dependencia obligatoria desde el ámbito.
Línea 33	<code>context.Database.EnsureDeleted();</code> → Reinicia la base de demostración para un E2E determinista.
Línea 34	<code>context.Database.Migrate();</code> → Aplica la historia real de migraciones heredada.
Línea 35	<code>DemoData.Seed(context);</code> → Carga el dataset determinista de AceriaData.
Línea 36	<code>context.ChangeTracker.Clear();</code> → Limpia tracking antes de la demostración.
Línea 37	<code>` `</code> → Separa bloques lógicos.

Línea 38	<code>var useCase = scope.ServiceProvider.GetRequiredService<TraduccionConsultasUseCase>();</code> → Resuelve una dependencia obligatoria desde el ámbito.
Línea 39	<code>useCase.Ejecutar();</code> → Participa directamente en el flujo validado del checkpoint: <code>useCase.Ejecutar();</code>
Línea 40	<code>` `</code> → Separa bloques lógicos.
Línea 41	<code>Console.WriteLine("4.7 OK");</code> → Publica evidencia observable en la consola.

Paso 7: Compilar

```
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

El build debe finalizar sin errores; el delta se propaga a los estados posteriores.

Paso 8: Ejecutar en LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

POWERSHELL

La salida debe terminar con 4.7 OK. Las aserciones internas fallan si la evidencia no coincide.

Paso 9: Diagnóstico técnico

Reto resuelto. Comparar el SQL con LOWER(columna) frente a comparación directa y explicar qué debe medirse en SQL Server.

No uses solo tiempo: revisa SQL/roundtrips, cardinalidad, columnas, tracking y frontera de materialización.

Paso 10: Cierre acumulativo

1. Confirma el marcador E2E.
2. Comprueba que EnsureCreated no aparece.
3. Conserva las migraciones heredadas.
4. Verifica que Application no depende de EF Core.
5. Compara con el checkpoint anterior y documenta el delta.

Errores comunes revisados

- Confundir una medición aislada con una conclusión de rendimiento.
- Materializar antes de terminar filtros o proyecciones sin intención.
- Aplicar una técnica por regla general en lugar de observar la consulta.
- Relajar una aserción para ocultar un fallo en vez de corregir su causa.

Trazabilidad con la práctica fuente

La práctica fuente de 4.7 incluía además los siguientes focos docentes:

- Filtros no traducibles y frontera cliente/servidor.
- Funciones aplicadas a columnas y posible pérdida de sargabilidad.
- Reescritura de expresiones y uso de collation cuando corresponda.

Tratamiento en el M4 definitivo. Se corrige la fuente: EF Core 8 no filtra silenciosamente en memoria dentro de Where. El checkpoint exige observar InvalidOperationException y solo después demuestra evaluación cliente explícita con AsEnumerable().

Reto de ampliación procedente de la fuente. Reescribe una validación de formato para usar operaciones traducibles y explica qué parte debe seguir ejecutándose en SQL.

Errores de la fuente que deben seguir siendo diagnosticables

- No afirmar que un Where no traducible se ejecuta automáticamente en memoria.
- No aplicar ToLower a la columna sin analizar el impacto sobre el índice.
- No ocultar una frontera cliente implícita: hacerla explícita.

Analogía operativa

Una frontera cliente explícita es sacar las piezas de la máquina y continuar manualmente: se puede hacer, pero debe ser consciente.

Resultado esperado

El checkpoint 4.7 queda ejecutable, trazado y reproducible, y sirve como baseline física del punto siguiente.

Conexión con el siguiente punto

El siguiente estado es 4.8 y parte físicamente de este checkpoint.

Punto 4.8 — Split Queries: cuándo y cómo usarlas

Contexto del proyecto

Este checkpoint continúa M04/PROYECTO/4.7. Conserva solución, capas, filtros, índices y migraciones; M4 no crea migraciones vacías.

Objetivo práctico

SplitQuery ejecuta varios comandos y puede evitar explosión cartesiana. No implica una transacción independiente por subconsulta. Sin aislamiento adecuado puede no existir una instantánea consistente frente a cambios concurrentes.

Paso 1: Abrir el checkpoint

```
cd M04/PROYECTO/4.8
dotnet restore AceriaData.sln
```

POWERSHELL

Paso 2: Comprobar migraciones

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

POWERSHELL

Debe seguir apareciendo M2_2_12_Architecture.

Paso 3: Identificar el delta docente

El archivo principal del delta es src/AceriaData.Infrastructure/Repositories/Rendimiento48.cs. El checkpoint conserva todo el estado anterior.

Paso 4: Implementar y estudiar Infrastructure

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public sealed partial class OrdenRepositorio
{
    private IQueryable<OrdenFabricacion> ConsultaDosColeccionesM4() =>
        _context.OrdenesFabricacion
```

CSHARP

```

        .AsNoTrackingWithIdentityResolution()
        .Include(o => o.Planchas)
        .Include(o => o.OrdenesAleaciones)
        .ThenInclude(oa => oa.Aleacion)
        .OrderBy(o => o.Id);

public SplitQueryMetricaDto MedirSingleQueryM4()
{
    SqlCommandCounterInterceptor.Instance.Reset();
    var ordenes = ConsultaDosColeccionesM4().AsSingleQuery().ToList();

    return new SplitQueryMetricaDto
    {
        Ordenes = ordenes.Count,
        ConsultasSql = checked((int)SqlCommandCounterInterceptor.Instance.Count),
        Planchas = ordenes.Sum(o => o.Planchas.Count),
        RelacionesAleacion = ordenes.Sum(o => o.OrdenesAleaciones.Count)
    };
}

public SplitQueryMetricaDto MedirSplitQueryM4()
{
    SqlCommandCounterInterceptor.Instance.Reset();
    var ordenes = ConsultaDosColeccionesM4().AsSplitQuery().ToList();

    return new SplitQueryMetricaDto
    {
        Ordenes = ordenes.Count,
        ConsultasSql = checked((int)SqlCommandCounterInterceptor.Instance.Count),
        Planchas = ordenes.Sum(o => o.Planchas.Count),
        RelacionesAleacion = ordenes.Sum(o => o.OrdenesAleaciones.Count)
    };
}

public string ObtenerSqlSingleQueryM4() =>
    ConsultaDosColeccionesM4().AsSingleQuery().ToQueryString();

public string ObtenerSqlSplitQueryM4() =>
    ConsultaDosColeccionesM4().AsSplitQuery().ToQueryString();
}

```

Application no recibe DbContext, IQueryable ni referencias a Microsoft.EntityFrameworkCore.

Paso 5: Implementar el caso de uso

CSHARP

```

using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class SplitQueriesUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public SplitQueriesUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== 4.8 SINGLE QUERY VS SPLIT QUERY ===");
        var single = _unidad.Ordenes.MedirSingleQueryM4();
        var split = _unidad.Ordenes.MedirSplitQueryM4();

        if (single.ConsultasSql != 1)
            throw new InvalidOperationException($"4.8: SingleQuery ejecuto {single.ConsultasSql} comandos.");
        if (split.ConsultasSql != 3)
            throw new InvalidOperationException(
                $"4.8: SplitQuery ejecuto {split.ConsultasSql} comandos; se esperaban 3.");
        if (single.Ordenes != split.Ordenes ||
            single.Planchas != split.Planchas ||
            single.RelacionesAleacion != split.RelacionesAleacion)
            throw new InvalidOperationException("4.8: SingleQuery y SplitQuery no materializaron el mismo grafo.");

        Console.WriteLine($"SingleQuery: {single.ConsultasSql} comando SQL.");
        Console.WriteLine($"SplitQuery: {split.ConsultasSql} comandos SQL.");
        Console.WriteLine("--- ToQueryString SingleQuery ---");
        Console.WriteLine(_unidad.Ordenes.ObtenerSqlSingleQueryM4());
        Console.WriteLine("--- ToQueryString SplitQuery ---");
        Console.WriteLine(_unidad.Ordenes.ObtenerSqlSplitQueryM4());
    }
}

```

Explicación línea a línea — SplitQueriesUseCase.cs

Línea 1	<code>using AceriaData.Application.Interfaces;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	<code>` `</code> → Separa bloques lógicos.
Línea 3	<code>namespace AceriaData.Application.UseCases;</code> → Sitúa el archivo en la capa y espacio de nombres correspondiente.
Línea 4	<code>` `</code> → Separa bloques lógicos.
Línea 5	<code>public sealed class SplitQueriesUseCase</code> → Declara la clase concreta usada por el checkpoint.
Línea 6	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 7	<code>private readonly IUnidadDeTrabajo _unidad;</code> → Declara la dependencia conservada por la instancia.
Línea 8	<code>public SplitQueriesUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;</code> → Constructor del caso de uso e inyección de la unidad de trabajo.
Línea 9	<code>` `</code> → Separa bloques lógicos.
Línea 10	<code>public void Ejecutar()</code> → Define el flujo principal validado por el E2E.
Línea 11	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 12	<code>Console.WriteLine("=== 4.8 SINGLE QUERY VS SPLIT QUERY ===");</code> → Publica evidencia observable en la consola.
Línea 13	<code>var single = _unidad.Ordenes.MedirSingleQueryM4();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 14	<code>var split = _unidad.Ordenes.MedirSplitQueryM4();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 15	<code>` `</code> → Separa bloques lógicos.
Línea 16	<code>if (single.ConsultasSql != 1)</code> → Comprueba una condición contractual del E2E.
Línea 17	<code>throw new InvalidOperationException(\$"4.8: SingleQuery ejecuto {single.ConsultasSql} comandos.");</code> → Hace fallar el checkpoint si la evidencia no coincide.
Línea 18	<code>if (split.ConsultasSql != 3)</code> → Comprueba una condición contractual del E2E.
Línea 19	<code>throw new InvalidOperationException(</code> → Hace fallar el checkpoint si la evidencia no coincide.
Línea 20	<code> \$"4.8: SplitQuery ejecuto {split.ConsultasSql} comandos; se esperaban 3.");</code> → Participa directamente en el flujo validado del checkpoint: <code> \$"4.8: SplitQuery ejecuto {split.ConsultasSql} comandos; se esperaban 3.");</code>
Línea 21	<code>if (single.Ordenes != split.Ordenes </code> → Comprueba una condición contractual del E2E.
Línea 22	<code>single.Planchas != split.Planchas </code> → Participa directamente en el flujo validado del checkpoint: <code>single.Planchas != split.Planchas </code>
Línea 23	<code>single.RelacionesAleacion != split.RelacionesAleacion)</code> → Participa directamente en el flujo validado del checkpoint: <code>single.RelacionesAleacion != split.RelacionesAleacion)</code>
Línea 24	<code>throw new InvalidOperationException("4.8: SingleQuery y SplitQuery no materializaron el mismo grafo.");</code> → Hace fallar el checkpoint si la evidencia no coincide.

Línea 25	<code>` `</code> → Separa bloques lógicos.
Línea 26	<code>Console.WriteLine(\$"SingleQuery: {single.ConsultasSql} comando SQL.");</code> → Publica evidencia observable en la consola.
Línea 27	<code>Console.WriteLine(\$"SplitQuery: {split.ConsultasSql} comandos SQL.");</code> → Publica evidencia observable en la consola.
Línea 28	<code>Console.WriteLine("--- ToQueryString SingleQuery ---");</code> → Publica evidencia observable en la consola.
Línea 29	<code>Console.WriteLine(_unidad.Ordenes.ObtenerSqlSingleQueryM4());</code> → Publica evidencia observable en la consola.
Línea 30	<code>Console.WriteLine("--- ToQueryString SplitQuery ---");</code> → Publica evidencia observable en la consola.
Línea 31	<code>Console.WriteLine(_unidad.Ordenes.ObtenerSqlSplitQueryM4());</code> → Publica evidencia observable en la consola.
Línea 32	<code>}</code> → Delimita el bloque sintáctico asociado.
Línea 33	<code>}</code> → Delimita el bloque sintáctico asociado.

Paso 6: Preparar el composition root

CSHARP

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB");
?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<BuenasPracticasUseCase>();
services.AddScoped<AnalisisSqlUseCase>();
services.AddScoped<TrackingUseCase>();
services.AddScoped<IdentityResolutionUseCase>();
services.AddScoped<NMasUnoUseCase>();
services.AddScoped<SolucionesNMasUnoUseCase>();
services.AddScoped<OverFetchingUseCase>();
services.AddScoped<TraduccionConsultasUseCase>();
services.AddScoped<SplitQueriesUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true,
ValidateScopes = true });
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<SplitQueriesUseCase>();
useCase.Ejecutar();

Console.WriteLine("4.8 OK");
```

Explicación línea a línea — Program.cs

Línea 1	<code>using AceriaData.Application.UseCases;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	<code>using AceriaData.ConsoleApp;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 3	<code>using AceriaData.Infrastructure;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 4	<code>using AceriaData.Infrastructure.Persistence;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 5	<code>using Microsoft.EntityFrameworkCore;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 6	<code>using Microsoft.Extensions.Configuration;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 7	<code>using Microsoft.Extensions.DependencyInjection;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 8	<code>``</code> → Separa bloques lógicos.
Línea 9	<code>var configuration = new ConfigurationBuilder()</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 10	<code>.SetBasePath(AppContext.BaseDirectory)</code> → Participa directamente en el flujo validado del checkpoint: <code>.SetBasePath(AppContext.BaseDirectory)</code>
Línea 11	<code>.AddJsonFile("appsettings.json", optional: false)</code> → Participa directamente en el flujo validado del checkpoint: <code>.AddJsonFile("appsettings.json", optional: false)</code>
Línea 12	<code>.AddEnvironmentVariables()</code> → Participa directamente en el flujo validado del checkpoint: <code>.AddEnvironmentVariables()</code>
Línea 13	<code>.Build();</code> → Participa directamente en el flujo validado del checkpoint: <code>.Build();</code>
Línea 14	<code>``</code> → Separa bloques lógicos.
Línea 15	<code>var connectionString = configuration.GetConnectionString("AceriaDB")</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 16	<code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");</code> → Participa directamente en el flujo validado del checkpoint: <code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");</code>
Línea 17	<code>``</code> → Separa bloques lógicos.
Línea 18	<code>var services = new ServiceCollection();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 19	<code>services.AddAceriaInfrastructure(connectionString);</code> → Participa directamente en el flujo validado del checkpoint: <code>services.AddAceriaInfrastructure(connectionString);</code>
Línea 20	<code>services.AddScoped<BuenasPracticasUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 21	<code>services.AddScoped<AnalisisSqlUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 22	<code>services.AddScoped<TrackingUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.

Línea 23	<code>services.AddScoped<IdentityResolutionUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 24	<code>services.AddScoped<NMasUnoUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 25	<code>services.AddScoped<SolucionesNMasUnoUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 26	<code>services.AddScoped<OverFetchingUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 27	<code>services.AddScoped<TraduccionConsultasUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 28	<code>services.AddScoped<SplitQueriesUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 29	<code>` `</code> → Separa bloques lógicos.
Línea 30	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true, ValidateScopes = true });</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 31	<code>using var scope = provider.CreateScope();</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 32	<code>` `</code> → Separa bloques lógicos.
Línea 33	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve una dependencia obligatoria desde el ámbito.
Línea 34	<code>context.Database.EnsureDeleted();</code> → Reinicia la base de demostración para un E2E determinista.
Línea 35	<code>context.Database.Migrate();</code> → Aplica la historia real de migraciones heredada.
Línea 36	<code>DemoData.Seed(context);</code> → Carga el dataset determinista de AceriaData.
Línea 37	<code>context.ChangeTracker.Clear();</code> → Limpia tracking antes de la demostración.
Línea 38	<code>` `</code> → Separa bloques lógicos.
Línea 39	<code>var useCase = scope.ServiceProvider.GetRequiredService<SplitQueriesUseCase>();</code> → Resuelve una dependencia obligatoria desde el ámbito.
Línea 40	<code>useCase.Ejecutar();</code> → Participa directamente en el flujo validado del checkpoint: <code>useCase.Ejecutar();</code>
Línea 41	<code>` `</code> → Separa bloques lógicos.
Línea 42	<code>Console.WriteLine("4.8 OK");</code> → Publica evidencia observable en la consola.

Paso 7: Compilar

```
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

El build debe finalizar sin errores; el delta se propaga a los estados posteriores.

Paso 8: Ejecutar en LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

POWERSHELL

La salida debe terminar con 4.8 OK. Las aserciones internas fallan si la evidencia no coincide.

Paso 9: Diagnóstico técnico

Reto resuelto. Explicar por qué dos colecciones multiplican filas en SingleQuery y por qué SplitQuery intercambia volumen por roundtrips.

No uses solo tiempo: revisa SQL/roundtrips, cardinalidad, columnas, tracking y frontera de materialización.

Paso 10: Cierre acumulativo

1. Confirma el marcador E2E.
2. Comprueba que EnsureCreated no aparece.
3. Conserva las migraciones heredadas.
4. Verifica que Application no depende de EF Core.
5. Compara con el checkpoint anterior y documenta el delta.

Errores comunes revisados

- Confundir una medición aislada con una conclusión de rendimiento.
- Materializar antes de terminar filtros o proyecciones sin intención.
- Aplicar una técnica por regla general en lugar de observar la consulta.
- Relajar una aserción para ocultar un fallo en vez de corregir su causa.

Trazabilidad con la práctica fuente

La práctica fuente de 4.8 incluía además los siguientes focos docentes:

- AsSingleQuery frente a AsSplitQuery con varias colecciones.
- Explosión cartesiana, duplicación de datos y roundtrips.
- Coherencia entre varios comandos y configuración global de Split Queries.

Tratamiento en el M4 definitivo. La configuración global se conserva como contenido de estudio, pero no se activa en la baseline porque ocultaría la comparación docente. La coherencia se explica en términos de aislamiento/transacción, no como una transacción independiente por subconsulta.

Reto de ampliación procedente de la fuente. Analiza cómo cambiaría el comportamiento si SplitQuery fuera global y qué advertencias querrías convertir en señal de diagnóstico.

Errores de la fuente que deben seguir siendo diagnosticables

- No afirmar que cada subconsulta de SplitQuery crea su propia transacción.
- No afirmar que una sola colección nunca puede beneficiarse; evaluar volumen y roundtrips.
- No comparar Single/Split con grafos distintos.

Analogía operativa

SingleQuery mezcla lotes en una hoja grande; SplitQuery los trae por separado y los ensambla por claves.

Resultado esperado

El checkpoint 4.8 queda ejecutable, trazado y reproducible, y sirve como baseline física del punto siguiente.

Conexión con el siguiente punto

El siguiente estado es 4.9 y parte físicamente de este checkpoint.

Punto 4.9 — Compiled Queries

Contexto del proyecto

Este checkpoint continúa M04/PROYECTO/4.8. Conserva solución, capas, filtros, índices y migraciones; M4 no crea migraciones vacías.

Objetivo práctico

EF Core ya cachea consultas por forma. EF.CompileQuery evita parte del trabajo de búsqueda y preparación de EF; no almacena el plan de ejecución de SQL Server. Debe medirse en hot paths y no se exige ganar una microprueba aislada.

Paso 1: Abrir el checkpoint

```
cd M04/PROYECTO/4.9
dotnet restore AceriaData.sln
```

POWERSHELL

Paso 2: Comprobar migraciones

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

POWERSHELL

Debe seguir apareciendo M2_2_12_Architecture.

Paso 3: Identificar el delta docente

El archivo principal del delta es src/AceriaData.Infrastructure/Repositories/Rendimiento49.cs. El checkpoint conserva todo el estado anterior.

Paso 4: Implementar y estudiar Infrastructure

```
using AceriaData.Domain.Entities;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public sealed partial class OrdenRepositorio
{
    private static readonly Func<AceriaDbContext, string, IEnumerable<OrdenFabricacion>>
        ConsultaCompiladaPorEstadoM4 =
        EF.CompileQuery(
            (AceriaDbContext context, string estado) =>
                context.OrdenesFabricacion
                    .AsNoTracking()
                    .Where(o => o.Estado == estado)
                    .OrderBy(o => o.FechaCreacion)
                    .ThenBy(o => o.Id)
                    .Select(o => o));

    public List<OrdenFabricacion> ObtenerPorEstadoNormalM4(string estado) =>
        _context.OrdenesFabricacion
            .AsNoTracking()
            .Where(o => o.Estado == estado)
            .OrderBy(o => o.FechaCreacion)
            .ThenBy(o => o.Id)
            .ToList();

    public List<OrdenFabricacion> ObtenerPorEstadoCompiladoM4(string estado) =>
        ConsultaCompiladaPorEstadoM4(_context, estado).ToList();
}
```

CSHARP

Application no recibe DbContext, IQueryable ni referencias a Microsoft.EntityFrameworkCore.

Paso 5: Implementar el caso de uso

```
using System.Diagnostics;
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class CompiledQueriesUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public CompiledQueriesUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;
```

CSHARP

```

public void Ejecutar()
{
    Console.WriteLine("=== 4.9 COMPILED QUERIES ===");

    var normal = _unidad.Ordenes.ObtenerPorEstadoNormalM4("Pendiente");
    var compilada = _unidad.Ordenes.ObtenerPorEstadoCompiladoM4("Pendiente");

    if (!normal.Select(o => o.Id).SequenceEqual(compilada.Select(o => o.Id)))
        throw new InvalidOperationException("4.9: consulta normal y compilada no son equivalentes.");

    var swNormal = Stopwatch.StartNew();
    for (var i = 0; i < 20; i++)
        _ = _unidad.Ordenes.ObtenerPorEstadoNormalM4("Pendiente");
    swNormal.Stop();

    var swCompilada = Stopwatch.StartNew();
    for (var i = 0; i < 20; i++)
        _ = _unidad.Ordenes.ObtenerPorEstadoCompiladoM4("Pendiente");
    swCompilada.Stop();

    Console.WriteLine($"Normal: {swNormal.ElapsedTicks} ticks | Compilada: {swCompilada.ElapsedTicks} ticks");
    Console.WriteLine(
        "Medicion observacional: no se exige que la compiled query gane en un dataset pequeno; se valida
    equivalencia y reutilizacion.");
}
}

```

Explicación línea a línea — CompiledQueriesUseCase.cs

Línea 1	using System.Diagnostics; → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	using AceriaData.Application.Interfaces; → Importa tipos o extensiones requeridos por esta implementación.
Línea 3	`` → Separa bloques lógicos.
Línea 4	namespace AceriaData.Application.UseCases; → Sitúa el archivo en la capa y espacio de nombres correspondiente.
Línea 5	`` → Separa bloques lógicos.
Línea 6	public sealed class CompiledQueriesUseCase → Declara la clase concreta usada por el checkpoint.
Línea 7	{ → Delimita el bloque sintáctico asociado.
Línea 8	private readonly IUnidadDeTrabajo _unidad; → Declara la dependencia conservada por la instancia.
Línea 9	public CompiledQueriesUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad; → Constructor del caso de uso e inyección de la unidad de trabajo.
Línea 10	`` → Separa bloques lógicos.
Línea 11	public void Ejecutar() → Define el flujo principal validado por el E2E.
Línea 12	{ → Delimita el bloque sintáctico asociado.
Línea 13	Console.WriteLine("=== 4.9 COMPILED QUERIES ==="); → Publica evidencia observable en la consola.
Línea 14	`` → Separa bloques lógicos.
Línea 15	var normal = _unidad.Ordenes.ObtenerPorEstadoNormalM4("Pendiente"); → Calcula y conserva el resultado que será validado o mostrado.
Línea 16	var compilada = _unidad.Ordenes.ObtenerPorEstadoCompiladoM4("Pendiente"); → Calcula y conserva el resultado que será validado o mostrado.
Línea 17	`` → Separa bloques lógicos.

Línea 18	<code>if (!normal.Select(o => o.Id).SequenceEqual(compilada.Select(o => o.Id)))</code> → Proyecta la forma de resultado y controla datos materializados.
Línea 19	<code>throw new InvalidOperationException("4.9: consulta normal y compilada no son equivalentes.");</code> → Hace fallar el checkpoint si la evidencia no coincide.
Línea 20	<code>` `</code> → Separa bloques lógicos.
Línea 21	<code>var swNormal = Stopwatch.StartNew();</code> → Participa en la medición temporal observacional.
Línea 22	<code>for (var i = 0; i < 20; i++)</code> → Participa directamente en el flujo validado del checkpoint: <code>for (var i = 0; i < 20; i++)</code>
Línea 23	<code>_ = _unidad.Ordenes.ObtenerPorEstadoNormalM4("Pendiente");</code> → Participa directamente en el flujo validado del checkpoint: <code>_ = _unidad.Ordenes.ObtenerPorEstadoNormalM4("Pendiente");</code>
Línea 24	<code>swNormal.Stop();</code> → Participa directamente en el flujo validado del checkpoint: <code>swNormal.Stop();</code>
Línea 25	<code>` `</code> → Separa bloques lógicos.
Línea 26	<code>var swCompilada = Stopwatch.StartNew();</code> → Participa en la medición temporal observacional.
Línea 27	<code>for (var i = 0; i < 20; i++)</code> → Participa directamente en el flujo validado del checkpoint: <code>for (var i = 0; i < 20; i++)</code>
Línea 28	<code>_ = _unidad.Ordenes.ObtenerPorEstadoCompiladoM4("Pendiente");</code> → Participa directamente en el flujo validado del checkpoint: <code>_ = _unidad.Ordenes.ObtenerPorEstadoCompiladoM4("Pendiente");</code>
Línea 29	<code>swCompilada.Stop();</code> → Participa directamente en el flujo validado del checkpoint: <code>swCompilada.Stop();</code>
Línea 30	<code>` `</code> → Separa bloques lógicos.
Línea 31	<code>Console.WriteLine(\$"Normal: {swNormal.ElapsedTicks} ticks Compilada: {swCompilada.ElapsedTicks} ticks");</code> → Publica evidencia observable en la consola.
Línea 32	<code>Console.WriteLine(</code> → Publica evidencia observable en la consola.
Línea 33	"Medicion observacional: no se exige que la compiled query gane en un dataset pequeno; se valida equivalencia y reutilizacion."); → Participa directamente en el flujo validado del checkpoint: "Medicion observacional: no se exige que la compiled query gane en un dataset pequeno; se valida equivalencia y reutilizacion.");
Línea 34	<code>}</code> → Delimita el bloque sintáctico asociado.
Línea 35	<code>}</code> → Delimita el bloque sintáctico asociado.

Paso 6: Preparar el composition root

CSHARP

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");
```

```

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<BuenasPracticasUseCase>();
services.AddScoped<AnalisisSqlUseCase>();
services.AddScoped<TrackingUseCase>();
services.AddScoped<IdentityResolutionUseCase>();
services.AddScoped<NMasUnoUseCase>();
services.AddScoped<SolucionesNMasUnoUseCase>();
services.AddScoped<OverFetchingUseCase>();
services.AddScoped<TraduccionConsultasUseCase>();
services.AddScoped<SplitQueriesUseCase>();
services.AddScoped<CompiledQueriesUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true,
ValidateScopes = true });
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<CompiledQueriesUseCase>();
useCase.Ejecutar();

Console.WriteLine("4.9 OK");

```

Explicación línea a línea — Program.cs

Línea 1	using AceriaData.Application.UseCases; → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	using AceriaData.ConsoleApp; → Importa tipos o extensiones requeridos por esta implementación.
Línea 3	using AceriaData.Infrastructure; → Importa tipos o extensiones requeridos por esta implementación.
Línea 4	using AceriaData.Infrastructure.Persistence; → Importa tipos o extensiones requeridos por esta implementación.
Línea 5	using Microsoft.EntityFrameworkCore; → Importa tipos o extensiones requeridos por esta implementación.
Línea 6	using Microsoft.Extensions.Configuration; → Importa tipos o extensiones requeridos por esta implementación.
Línea 7	using Microsoft.Extensions.DependencyInjection; → Importa tipos o extensiones requeridos por esta implementación.
Línea 8	`` → Separa bloques lógicos.
Línea 9	var configuration = new ConfigurationBuilder() → Calcula y conserva el resultado que será validado o mostrado.
Línea 10	.SetBasePath(AppContext.BaseDirectory) → Participa directamente en el flujo validado del checkpoint: .SetBasePath(AppContext.BaseDirectory)
Línea 11	.AddJsonFile("appsettings.json", optional: false) → Participa directamente en el flujo validado del checkpoint: .AddJsonFile("appsettings.json", optional: false)
Línea 12	.AddEnvironmentVariables() → Participa directamente en el flujo validado del checkpoint: .AddEnvironmentVariables()
Línea 13	.Build(); → Participa directamente en el flujo validado del checkpoint: .Build();
Línea 14	`` → Separa bloques lógicos.

Línea 15	<code>var connectionString = configuration.GetConnectionString("AceriaDB");</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 16	<code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");</code> → Participa directamente en el flujo validado del checkpoint: <code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");</code>
Línea 17	<code>``</code> → Separa bloques lógicos.
Línea 18	<code>var services = new ServiceCollection();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 19	<code>services.AddAceriaInfrastructure(connectionString);</code> → Participa directamente en el flujo validado del checkpoint: <code>services.AddAceriaInfrastructure(connectionString);</code>
Línea 20	<code>services.AddScoped<BuenasPracticasUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 21	<code>services.AddScoped<AnalisisSqlUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 22	<code>services.AddScoped<TrackingUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 23	<code>services.AddScoped<IdentityResolutionUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 24	<code>services.AddScoped<NMasUnoUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 25	<code>services.AddScoped<SolucionesNMasUnoUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 26	<code>services.AddScoped<OverFetchingUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 27	<code>services.AddScoped<TraduccionConsultasUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 28	<code>services.AddScoped<SplitQueriesUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 29	<code>services.AddScoped<CompiledQueriesUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 30	<code>``</code> → Separa bloques lógicos.
Línea 31	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true, ValidateScopes = true });</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 32	<code>using var scope = provider.CreateScope();</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 33	<code>``</code> → Separa bloques lógicos.
Línea 34	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve una dependencia obligatoria desde el ámbito.
Línea 35	<code>context.Database.EnsureDeleted();</code> → Reinicia la base de demostración para un E2E determinista.
Línea 36	<code>context.Database.Migrate();</code> → Aplica la historia real de migraciones heredada.
Línea 37	<code>DemoData.Seed(context);</code> → Carga el dataset determinista de AceriaData.
Línea 38	<code>context.ChangeTracker.Clear();</code> → Limpia tracking antes de la demostración.

Línea 39	<code>` `</code> → Separa bloques lógicos.
Línea 40	<code>var useCase = scope.ServiceProvider.GetRequiredService<CompiledQueriesUseCase>();</code> → Resuelve una dependencia obligatoria desde el ámbito.
Línea 41	<code>useCase.Ejecutar();</code> → Participa directamente en el flujo validado del checkpoint: <code>useCase.Ejecutar();</code>
Línea 42	<code>` `</code> → Separa bloques lógicos.
Línea 43	<code>Console.WriteLine("4.9 OK");</code> → Publica evidencia observable en la consola.

Paso 7: Compilar

```
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

El build debe finalizar sin errores; el delta se propaga a los estados posteriores.

Paso 8: Ejecutar en LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

POWERSHELL

La salida debe terminar con 4.9 OK. Las aserciones internas fallan si la evidencia no coincide.

Paso 9: Diagnóstico técnico

Reto resuelto. Justificar cuándo el coste evitado por `CompileQuery` puede importar frente a red y base de datos.

No uses solo tiempo: revisa SQL/roundtrips, cardinalidad, columnas, tracking y frontera de materialización.

Paso 10: Cierre acumulativo

1. Confirma el marcador E2E.
2. Comprueba que `EnsureCreated` no aparece.
3. Conserva las migraciones heredadas.
4. Verifica que `Application` no depende de EF Core.
5. Compara con el checkpoint anterior y documenta el delta.

Errores comunes revisados

- Confundir una medición aislada con una conclusión de rendimiento.
- Materializar antes de terminar filtros o proyecciones sin intención.
- Aplicar una técnica por regla general en lugar de observar la consulta.
- Relajar una aserción para ocultar un fallo en vez de corregir su causa.

Trazabilidad con la práctica fuente

La práctica fuente de 4.9 incluía además los siguientes focos docentes:

- EF.CompileQuery y EF.CompileAsyncQuery, parámetros y proyecciones.
- Caché interna de consultas de EF Core y coste que realmente evita una compiled query.
- Medición en hot paths sin prometer una mejora universal.

Tratamiento en el M4 definitivo. El checkpoint ejecutable usa una compiled query síncrona parametrizada para validar equivalencia. Async, proyección y variantes se conservan en teoría y como ampliación, sin inventar una ventaja temporal obligatoria.

Reto de ampliación procedente de la fuente. Diseña una compiled query proyectada y explica qué coste de EF evita frente al coste de red y SQL Server.

Errores de la fuente que deben seguir siendo diagnosticables

- No compilar el delegado en cada llamada.
- No afirmar que EF.CompileQuery almacena el plan de ejecución de SQL Server.
- No usar un umbral de tiempo como condición de éxito del E2E.

Analogía operativa

CompiledQuery guarda una ruta de preparación en EF; no reserva una vía dentro de SQL Server.

Resultado esperado

El checkpoint 4.9 queda ejecutable, trazado y reproducible, y sirve como baseline física del punto siguiente.

Conexión con el siguiente punto

El siguiente estado es 4.10 y parte físicamente de este checkpoint.

Punto 4.10 — Paginación eficiente: Skip/Take y keyset pagination

Contexto del proyecto

Este checkpoint continúa M04/PROYECTO/4.9. Conserva solución, capas, filtros, índices y migraciones; M4 no crea migraciones vacías.

Objetivo práctico

Toda paginación necesita orden totalmente determinista. AceriaData ordena por FechaCreacion e Id; keyset usa ambos valores como cursor. Offset es válido para saltos arbitrarios pero puede encarecerse con offsets altos.

Paso 1: Abrir el checkpoint

```
cd M04/PROYECTO/4.10
dotnet restore AceriaData.sln
```

POWERSHELL

Paso 2: Comprobar migraciones

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

POWERSHELL

Debe seguir apareciendo M2_2_12_Architecture.

Paso 3: Identificar el delta docente

El archivo principal del delta es src/AceriaData.Infrastructure/Repositories/Rendimiento410.cs. El checkpoint conserva todo el estado anterior.

Paso 4: Implementar y estudiar Infrastructure

```
using AceriaData.Application.Dtos;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public sealed partial class OrdenRepository
{
    public PaginaOrdenesDto ObtenerPaginaOffsetM4(int pagina, int tamano)
    {
        if (pagina < 1) throw new ArgumentOutOfRangeException(nameof(pagina));
    }
}
```

CSHARP

```

        if (tamano < 1) throw new ArgumentOutOfRangeException(nameof(tamano));

        var consulta = _context.OrdenesFabricacion
            .AsNoTracking()
            .OrderBy(o => o.FechaCreacion)
            .ThenBy(o => o.Id)
            .Skip((pagina - 1) * tamano)
            .Take(tamano)
            .Select(o => new OrdenPaginaDto
            {
                Id = o.Id,
                NumeroOrden = o.NumeroOrden,
                Cliente = o.Cliente,
                Estado = o.Estado,
                FechaCreacion = o.FechaCreacion
            });

        return new PaginaOrdenesDto
        {
            Sql = consulta.ToQueryString(),
            Elementos = consulta.ToList()
        };
    }

    public PaginaOrdenesDto ObtenerPaginaKeysetM4(
        DateTime ultimaFecha,
        int ultimoId,
        int tamano)
    {
        if (tamano < 1) throw new ArgumentOutOfRangeException(nameof(tamano));

        var consulta = _context.OrdenesFabricacion
            .AsNoTracking()
            .Where(o =>
                o.FechaCreacion > ultimaFecha ||
                (o.FechaCreacion == ultimaFecha && o.Id > ultimoId))
            .OrderBy(o => o.FechaCreacion)
            .ThenBy(o => o.Id)
            .Take(tamano)
            .Select(o => new OrdenPaginaDto
            {
                Id = o.Id,
                NumeroOrden = o.NumeroOrden,
                Cliente = o.Cliente,
                Estado = o.Estado,
                FechaCreacion = o.FechaCreacion
            });

        return new PaginaOrdenesDto
        {
            Sql = consulta.ToQueryString(),
            Elementos = consulta.ToList()
        };
    }
}

```

Archivo complementario: src/AceriaData.Console/DemoData.cs

CSHARP

```

using AceriaData.Domain.Entities;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.ConsoleApp;

public static class DemoData
{
    public static void Seed(AceriaDbContext context)
    {
        if (context.OrdenesFabricacion.IgnoreQueryFilters().Any()) return;

        var a36 = new Aleacion { Nombre = "ASTM A36",Codigo = "A36", PorcentajeCarbono = 0.20, PorcentajeManganeso = 0.80 };
        var s355 = new Aleacion { Nombre = "S355",Codigo = "S355", PorcentajeCarbono = 0.18, PorcentajeManganeso = 1.20 };

        var o1 = Orden("OF-2024-0001", "Constructora del Norte", "Pendiente", new DateTime(2024, 1, 15));
        o1.Planchas.Add(new PlanchaAcero { Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso = 370.5m });
        o1.Planchas.Add(new PlanchaAcero { Espesor = 12.0, Ancho = 1200, Largo = 2500, Peso = 280.8m });
        o1.Detalle = new DetalleOrden { ComposicionQuimica = "C 0.20%; Mn 0.80%", TemperaturaColada = 1540 };
        o1.Certificado = new CertificadoCalidad { NumeroCertificado = "CERT-0001", FechaEmision = new DateTime(2024, 1, 20), OrganismoCertificador = "Aceria QA" };
        o1.OrdenesAleaciones.Add(new OrdenAleacion { Aleacion = a36, CantidadUtilizada = 500m, FechaAsignacion = new DateTime(2024, 1, 15) });
    }
}

```

```

var o2 = Orden("OF-2024-0002", "Constructora del Sur", "Pendiente", new DateTime(2024, 2, 20));
o2.Planchas.Add(new PlanchaAcero { Espesor = 8.0, Ancho = 1000, Largo = 2000, Peso = 125.6m });
o2.Detalle = new DetalleOrden { ComposicionQuimica = "C 0.18%; Mn 1.20%", TemperaturaColada = 1535 };
o2.OrdenesAleaciones.Add(new OrdenAleacion { Aleacion = s355, CantidadUtilizada = 250m, FechaAsignacion =
new DateTime(2024, 2, 20) });

var o3 = Orden("OF-2024-0003", "Constructora del Norte", "EnProceso", new DateTime(2024, 3, 10));
o3.Planchas.Add(new PlanchaAcero { Espesor = 15.0, Ancho = 1800, Largo = 3500, Peso = 450.0m });
o3.Detalle = new DetalleOrden { ComposicionQuimica = "C 0.19%; Mn 1.10%", TemperaturaColada = 1545 };
o3.OrdenesAleaciones.Add(new OrdenAleacion { Aleacion = s355, CantidadUtilizada = 400m, FechaAsignacion =
new DateTime(2024, 3, 10) });

var o4 = Orden("OF-2024-0004", "Constructora del Norte", "Pendiente", new DateTime(2024, 4, 5));
o4.Detalle = new DetalleOrden { ComposicionQuimica = "C 0.17%; Mn 0.90%", TemperaturaColada = 1538 };

var o5 = Orden("OF-2024-0005", "Constructora del Este", "Completada", new DateTime(2024, 5, 12));
o5.Planchas.Add(new PlanchaAcero { Espesor = 9.0, Ancho = 1100, Largo = 2100, Peso = 200.0m });
o5.OrdenesAleaciones.Add(new OrdenAleacion { Aleacion = a36, CantidadUtilizada = 180m, FechaAsignacion = new
DateTime(2024, 5, 12) });

context.AddRange(a36, s355, o1, o2, o3, o4, o5);
context.SaveChanges();

var extras = Enumerable.Range(6, 15)
    .Select(i => Orden(
        $"OF-2024-{i:0000}",
        i % 3 == 0 ? "Constructora del Norte" :
        i % 3 == 1 ? "Constructora del Sur" : "Constructora del Este",
        i % 4 == 0 ? "EnProceso" : "Pendiente",
        new DateTime(2024, 6, 1).AddDays(i)))
    .ToList();

context.OrdenesFabricacion.AddRange(extras);
context.SaveChanges();
}

private static OrdenFabricacion Orden(string numero, string cliente, string estado, DateTime fecha) => new()
{
    NumeroOrden = numero, Cliente = cliente, Estado = estado, FechaCreacion = fecha
};
}

```

Application no recibe DbContext, IQueryable ni referencias a Microsoft.EntityFrameworkCore.

Paso 5: Implementar el caso de uso

CSHARP

```

using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class PaginacionUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public PaginacionUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== 4.10 PAGINACION ===");

        var offset = _unidad.Ordenes.ObtenerPaginaOffsetM4(2, 5);
        var primeraKeyset = _unidad.Ordenes.ObtenerPaginaKeysetM4(DateTime.MinValue, 0, 5);
        var cursor = primeraKeyset.Elementos.Last();
        var segundaKeyset = _unidad.Ordenes.ObtenerPaginaKeysetM4(
            cursor.FechaCreacion,
            cursor.Id,
            5);

        if (offset.Elementos.Count != 5 ||
            primeraKeyset.Elementos.Count != 5 ||
            segundaKeyset.Elementos.Count != 5)
            throw new InvalidOperationException("4.10: paginacion no devolvio el tamano esperado.");

        if (primeraKeyset.Elementos
            .Select(x => x.Id)
            .Intersect(segundaKeyset.Elementos.Select(x => x.Id))
            .Any())
            throw new InvalidOperationException("4.10: keyset repitio filas entre paginas.");

        Console.WriteLine("--- OFFSET ---");
        Console.WriteLine(offset.Sql);
    }
}

```

```

        Console.WriteLine("--- KEYSET ---");
        Console.WriteLine(segundaKeyset.Sql);
    }
}

```

Explicación línea a línea — PaginacionUseCase.cs

Línea 1	<code>using AceriaData.Application.Interfaces;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	<code>` `</code> → Separa bloques lógicos.
Línea 3	<code>namespace AceriaData.Application.UseCases;</code> → Sitúa el archivo en la capa y espacio de nombres correspondiente.
Línea 4	<code>` `</code> → Separa bloques lógicos.
Línea 5	<code>public sealed class PaginacionUseCase</code> → Declara la clase concreta usada por el checkpoint.
Línea 6	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 7	<code>private readonly IUnidadDeTrabajo _unidad;</code> → Declara la dependencia conservada por la instancia.
Línea 8	<code>public PaginacionUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;</code> → Constructor del caso de uso e inyección de la unidad de trabajo.
Línea 9	<code>` `</code> → Separa bloques lógicos.
Línea 10	<code>public void Ejecutar()</code> → Define el flujo principal validado por el E2E.
Línea 11	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 12	<code>Console.WriteLine("=== 4.10 PAGINACION ===");</code> → Publica evidencia observable en la consola.
Línea 13	<code>` `</code> → Separa bloques lógicos.
Línea 14	<code>var offset = _unidad.Ordenes.ObtenerPaginaOffsetM4(2, 5);</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 15	<code>var primeraKeyset = _unidad.Ordenes.ObtenerPaginaKeysetM4(DateTime.MinValue, 0, 5);</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 16	<code>var cursor = primeraKeyset.Elementos.Last();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 17	<code>var segundaKeyset = _unidad.Ordenes.ObtenerPaginaKeysetM4(</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 18	<code>cursor.FechaCreacion,</code> → Participa directamente en el flujo validado del checkpoint: <code>cursor.FechaCreacion,</code>
Línea 19	<code>cursor.Id,</code> → Participa directamente en el flujo validado del checkpoint: <code>cursor.Id,</code>
Línea 20	<code>5);</code> → Participa directamente en el flujo validado del checkpoint: <code>5);</code>
Línea 21	<code>` `</code> → Separa bloques lógicos.
Línea 22	<code>if (offset.Elementos.Count != 5 </code> → Comprueba una condición contractual del E2E.
Línea 23	<code>primeraKeyset.Elementos.Count != 5 </code> → Participa directamente en el flujo validado del checkpoint: <code>primeraKeyset.Elementos.Count != 5 </code>

Línea 24	<code>segundaKeyset.Elementos.Count != 5)</code> → Participa directamente en el flujo validado del checkpoint: <code>segundaKeyset.Elementos.Count != 5)</code>
Línea 25	<code>throw new InvalidOperationException("4.10: paginacion no devolvio el tamano esperado.");</code> → Hace fallar el checkpoint si la evidencia no coincide.
Línea 26	<code>` `</code> → Separa bloques lógicos.
Línea 27	<code>if (primeraKeyset.Elementos</code> → Comprueba una condición contractual del E2E.
Línea 28	<code>.Select(x => x.Id)</code> → Proyecta la forma de resultado y controla datos materializados.
Línea 29	<code>.Intersect(segundaKeyset.Elementos.Select(x => x.Id))</code> → Proyecta la forma de resultado y controla datos materializados.
Línea 30	<code>.Any()</code> → Participa directamente en el flujo validado del checkpoint: <code>.Any()</code>
Línea 31	<code>throw new InvalidOperationException("4.10: keyset repitio filas entre paginas.");</code> → Hace fallar el checkpoint si la evidencia no coincide.
Línea 32	<code>` `</code> → Separa bloques lógicos.
Línea 33	<code>Console.WriteLine("--- OFFSET ---");</code> → Publica evidencia observable en la consola.
Línea 34	<code>Console.WriteLine(offset.Sql);</code> → Publica evidencia observable en la consola.
Línea 35	<code>Console.WriteLine("--- KEYSET ---");</code> → Publica evidencia observable en la consola.
Línea 36	<code>Console.WriteLine(segundaKeyset.Sql);</code> → Publica evidencia observable en la consola.
Línea 37	<code>}</code> → Delimita el bloque sintáctico asociado.
Línea 38	<code>}</code> → Delimita el bloque sintáctico asociado.

Paso 6: Preparar el composition root

CSHARP

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<BuenasPracticasUseCase>();
services.AddScoped<AnalisisSqlUseCase>();
services.AddScoped<TrackingUseCase>();
services.AddScoped<IdentityResolutionUseCase>();
services.AddScoped<NMasUnoUseCase>();
services.AddScoped<SolucionesNMasUnoUseCase>();
services.AddScoped<OverFetchingUseCase>();
services.AddScoped<TraduccionConsultasUseCase>();
services.AddScoped<SplitQueriesUseCase>();
services.AddScoped<CompiledQueriesUseCase>();
services.AddScoped<PaginacionUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true,
    ValidateScopes = true });
```

```

using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<PaginacionUseCase>();
useCase.Ejecutar();

Console.WriteLine("4.10 OK");

```

Explicación línea a línea — Program.cs

Línea 1	using AceriaData.Application.UseCases; → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	using AceriaData.ConsoleApp; → Importa tipos o extensiones requeridos por esta implementación.
Línea 3	using AceriaData.Infrastructure; → Importa tipos o extensiones requeridos por esta implementación.
Línea 4	using AceriaData.Infrastructure.Persistence; → Importa tipos o extensiones requeridos por esta implementación.
Línea 5	using Microsoft.EntityFrameworkCore; → Importa tipos o extensiones requeridos por esta implementación.
Línea 6	using Microsoft.Extensions.Configuration; → Importa tipos o extensiones requeridos por esta implementación.
Línea 7	using Microsoft.Extensions.DependencyInjection; → Importa tipos o extensiones requeridos por esta implementación.
Línea 8	`` → Separa bloques lógicos.
Línea 9	var configuration = new ConfigurationBuilder() → Calcula y conserva el resultado que será validado o mostrado.
Línea 10	.SetBasePath(AppContext.BaseDirectory) → Participa directamente en el flujo validado del checkpoint: .SetBasePath(AppContext.BaseDirectory)
Línea 11	.AddJsonFile("appsettings.json", optional: false) → Participa directamente en el flujo validado del checkpoint: .AddJsonFile("appsettings.json", optional: false)
Línea 12	.AddEnvironmentVariables() → Participa directamente en el flujo validado del checkpoint: .AddEnvironmentVariables()
Línea 13	.Build(); → Participa directamente en el flujo validado del checkpoint: .Build();
Línea 14	`` → Separa bloques lógicos.
Línea 15	var connectionString = configuration.GetConnectionString("AceriaDB") → Calcula y conserva el resultado que será validado o mostrado.
Línea 16	?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB"); → Participa directamente en el flujo validado del checkpoint: ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");
Línea 17	`` → Separa bloques lógicos.
Línea 18	var services = new ServiceCollection(); → Calcula y conserva el resultado que será validado o mostrado.

Línea 19	<code>services.AddAceriaInfrastructure(connectionString);</code> → Participa directamente en el flujo validado del checkpoint: <code>services.AddAceriaInfrastructure(connectionString);</code>
Línea 20	<code>services.AddScoped<BuenasPracticasUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 21	<code>services.AddScoped<AnalisisSqlUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 22	<code>services.AddScoped<TrackingUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 23	<code>services.AddScoped<IdentityResolutionUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 24	<code>services.AddScoped<NMasUnoUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 25	<code>services.AddScoped<SolucionesNMasUnoUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 26	<code>services.AddScoped<OverFetchingUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 27	<code>services.AddScoped<TraduccionConsultasUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 28	<code>services.AddScoped<SplitQueriesUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 29	<code>services.AddScoped<CompiledQueriesUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 30	<code>services.AddScoped<PaginacionUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 31	<code>` `</code> → Separa bloques lógicos.
Línea 32	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true, ValidateScopes = true });</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 33	<code>using var scope = provider.CreateScope();</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 34	<code>` `</code> → Separa bloques lógicos.
Línea 35	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve una dependencia obligatoria desde el ámbito.
Línea 36	<code>context.Database.EnsureDeleted();</code> → Reinicia la base de demostración para un E2E determinista.
Línea 37	<code>context.Database.Migrate();</code> → Aplica la historia real de migraciones heredada.
Línea 38	<code>DemoData.Seed(context);</code> → Carga el dataset determinista de <code>AceriaData</code> .
Línea 39	<code>context.ChangeTracker.Clear();</code> → Limpia tracking antes de la demostración.
Línea 40	<code>` `</code> → Separa bloques lógicos.
Línea 41	<code>var useCase = scope.ServiceProvider.GetRequiredService<PaginacionUseCase>();</code> → Resuelve una dependencia obligatoria desde el ámbito.
Línea 42	<code>useCase.Ejecutar();</code> → Participa directamente en el flujo validado del checkpoint: <code>useCase.Ejecutar();</code>
Línea 43	<code>` `</code> → Separa bloques lógicos.

Línea 44

Console.WriteLine("4.10 OK"); → Publica evidencia observable en la consola.

Paso 7: Compilar

```
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

El build debe finalizar sin errores; el delta se propaga a los estados posteriores.

Paso 8: Ejecutar en LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

POWERSHELL

La salida debe terminar con 4.10 OK. Las aserciones internas fallan si la evidencia no coincide.

Paso 9: Diagnóstico técnico

Reto resuelto. Construir la condición seek para un orden compuesto FechaCreacion + Id.

No uses solo tiempo: revisa SQL/roundtrips, cardinalidad, columnas, tracking y frontera de materialización.

Paso 10: Cierre acumulativo

1. Confirma el marcador E2E.
2. Comprueba que EnsureCreated no aparece.
3. Conserva las migraciones heredadas.
4. Verifica que Application no depende de EF Core.
5. Compara con el checkpoint anterior y documenta el delta.

Errores comunes revisados

- Confundir una medición aislada con una conclusión de rendimiento.
- Materializar antes de terminar filtros o proyecciones sin intención.
- Aplicar una técnica por regla general en lugar de observar la consulta.
- Relajar una aserción para ocultar un fallo en vez de corregir su causa.

Trazabilidad con la práctica fuente

La práctica fuente de 4.10 incluía además los siguientes focos docentes:

- Offset pagination con Skip/Take.
- Keyset pagination con orden totalmente determinista.
- Filtro, proyección y dirección de paginación.

Tratamiento en el M4 definitivo. La fuente advertía del riesgo de usar solo fecha; el checkpoint lo corrige con cursor compuesto FechaCreacion + Id y añade datos suficientes para recorrer varias páginas.

Reto de ampliación procedente de la fuente. Añade mentalmente un filtro de estado a la paginación y conserva el mismo orden compuesto para no saltar ni repetir filas.

Errores de la fuente que deben seguir siendo diagnosticables

- No paginar sin OrderBy.
- No usar una clave de ordenación no única como cursor único.
- No dejar tracking activo para listados paginados de solo lectura.

Analogía operativa

Offset cuenta cajas desde el principio; keyset continúa desde la etiqueta exacta de la última caja vista.

Resultado esperado

El checkpoint 4.10 queda ejecutable, trazado y reproducible, y sirve como baseline física del punto siguiente.

Conexión con el siguiente punto

El siguiente estado es 4.11 y parte físicamente de este checkpoint.

Punto 4.11 — Diagnóstico con logs, métricas y herramientas

Contexto del proyecto

Este checkpoint continúa M04/PROYECTO/4.10. Conserva solución, capas, filtros, índices y migraciones; M4 no crea migraciones vacías.

Objetivo práctico

Un tiempo aislado no prueba rendimiento. El diagnóstico reproducible combina SQL, comandos, filas, tracking y tiempo, y usa TagWith para correlación.

Paso 1: Abrir el checkpoint

```
cd M04/PROYECTO/4.11
dotnet restore AceriaData.sln
```

POWERSHELL

Paso 2: Comprobar migraciones

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

POWERSHELL

Debe seguir apareciendo M2_2_12_Architecture.

Paso 3: Identificar el delta docente

El archivo principal del delta es src/AceriaData.Infrastructure/Repositories/Rendimiento411.cs. El checkpoint conserva todo el estado anterior.

Paso 4: Implementar y estudiar Infrastructure

```
using System.Diagnostics;
using AceriaData.Application.Dtos;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public sealed partial class OrdenRepository
{
    public DiagnosticoRendimientoDto DiagnosticarPendientesM4()
    {
        _context.ChangeTracker.Clear();
        SqlCommandCounterInterceptor.Instance.Reset();

        var consulta = _context.OrdenesFabricacion
            .TagWith("M4.11-DIAGNOSTICO")
            .AsNoTracking()
            .Where(o => o.Estado == "Pendiente")
            .OrderBy(o => o.FechaCreacion)
            .ThenBy(o => o.Id)
            .Select(o => new OrdenResumenDto
            {
                NumeroOrden = o.NumeroOrden,
                Cliente = o.Cliente,
                Estado = o.Estado,
            });
    }
}
```

CSHARP

```

        FechaCreacion = o.FechaCreacion
    })
    .Take(10);

    var sql = consulta.ToQueryString();

    var sw = Stopwatch.StartNew();
    var filas = consulta.ToList();
    sw.Stop();

    return new DiagnosticoRendimientoDto
    {
        Filas = filas.Count,
        ConsultasSql = checked((int)SqlCommandCounterInterceptor.Instance.Count),
        EntidadesRastreadas = _context.ChangeTracker.Entries().Count(),
        Ticks = sw.ElapsedTicks,
        Sql = sql
    };
}

```

Application no recibe DbContext, IQueryable ni referencias a Microsoft.EntityFrameworkCore.

Paso 5: Implementar el caso de uso

```

using AceriaData.Application.Interfaces;
namespace AceriaData.Application.UseCases;

public sealed class DiagnosticoRendimientoUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public DiagnosticoRendimientoUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== 4.11 DIAGNOSTICO DE RENDIMIENTO ===");
        var d = _unidad.Ordenes.DiagnosticarPendientesM4();

        if (d.Filas == 0 || d.ConsultasSql != 1 || d.EntidadesRastreadas != 0)
            throw new InvalidOperationException(
                "4.11: las metricas observables no coinciden con la consulta optimizada.");
        if (!d.Sql.Contains("M4.11-DIAGNOSTICO", StringComparison.Ordinal))
            throw new InvalidOperationException("4.11: falta TagWith en el SQL de diagnostico.");

        Console.WriteLine(
            $"Filas={d.Filas} | SQL commands={d.ConsultasSql} | Tracking={d.EntidadesRastreadas} | Ticks={d.Ticks}");
        Console.WriteLine(d.Sql);
    }
}

```

Explicación línea a línea — DiagnosticoRendimientoUseCase.cs

Línea 1	using AceriaData.Application.Interfaces; → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	`` → Separa bloques lógicos.
Línea 3	namespace AceriaData.Application.UseCases; → Sitúa el archivo en la capa y espacio de nombres correspondiente.
Línea 4	`` → Separa bloques lógicos.
Línea 5	public sealed class DiagnosticoRendimientoUseCase → Declara la clase concreta usada por el checkpoint.
Línea 6	{ → Delimita el bloque sintáctico asociado.
Línea 7	private readonly IUnidadDeTrabajo _unidad; → Declara la dependencia conservada por la instancia.

Línea 8	<code>public DiagnosticoRendimientoUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;</code> → Constructor del caso de uso e inyección de la unidad de trabajo.
Línea 9	<code>` `</code> → Separa bloques lógicos.
Línea 10	<code>public void Ejecutar()</code> → Define el flujo principal validado por el E2E.
Línea 11	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 12	<code>Console.WriteLine("=== 4.11 DIAGNOSTICO DE RENDIMIENTO ===");</code> → Publica evidencia observable en la consola.
Línea 13	<code>var d = _unidad.Ordenes.DiagnosticarPendientesM4();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 14	<code>` `</code> → Separa bloques lógicos.
Línea 15	<code>if (d.Filas == 0 d.ConsultasSql != 1 d.EntidadesRastreadas != 0)</code> → Comprueba una condición contractual del E2E.
Línea 16	<code>throw new InvalidOperationException(</code> → Hace fallar el checkpoint si la evidencia no coincide.
Línea 17	<code>"4.11: las metricas observables no coinciden con la consulta optimizada.");</code> → Participa directamente en el flujo validado del checkpoint: "4.11: las metricas observables no coinciden con la consulta optimizada.");
Línea 18	<code>if (!d.Sql.Contains("M4.11-DIAGNOSTICO", StringComparison.Ordinal))</code> → Comprueba una condición contractual del E2E.
Línea 19	<code>throw new InvalidOperationException("4.11: falta TagWith en el SQL de diagnostico.");</code> → Etiqueta el SQL para correlacionarlo con logs.
Línea 20	<code>` `</code> → Separa bloques lógicos.
Línea 21	<code>Console.WriteLine(</code> → Publica evidencia observable en la consola.
Línea 22	<code> \$"Filas={d.Filas} SQL commands={d.ConsultasSql} Tracking={d.EntidadesRastreadas} Ticks={d.Ticks}");</code> → Participa directamente en el flujo validado del checkpoint: <code> \$"Filas={d.Filas} SQL commands={d.ConsultasSql} Tracking={d.EntidadesRastreadas} Ticks={d.Ticks}");</code>
Línea 23	<code>Console.WriteLine(d.Sql);</code> → Publica evidencia observable en la consola.
Línea 24	<code>}</code> → Delimita el bloque sintáctico asociado.
Línea 25	<code>}</code> → Delimita el bloque sintáctico asociado.

Paso 6: Preparar el composition root

CSHARP

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");
```

```

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<BuenasPracticasUseCase>();
services.AddScoped<AnalisisSqlUseCase>();
services.AddScoped<TrackingUseCase>();
services.AddScoped<IdentityResolutionUseCase>();
services.AddScoped<NMasUnoUseCase>();
services.AddScoped<SolucionesNMasUnoUseCase>();
services.AddScoped<OverFetchingUseCase>();
services.AddScoped<TraduccionConsultasUseCase>();
services.AddScoped<SplitQueriesUseCase>();
services.AddScoped<CompiledQueriesUseCase>();
services.AddScoped<PaginacionUseCase>();
services.AddScoped<DiagnosticoRendimientoUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true,
ValidateScopes = true });
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<DiagnosticoRendimientoUseCase>();
useCase.Ejecutar();

Console.WriteLine("4.11 OK");

```

Explicación línea a línea — Program.cs

Línea 1	using AceriaData.Application.UseCases; → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	using AceriaData.ConsoleApp; → Importa tipos o extensiones requeridos por esta implementación.
Línea 3	using AceriaData.Infrastructure; → Importa tipos o extensiones requeridos por esta implementación.
Línea 4	using AceriaData.Infrastructure.Persistence; → Importa tipos o extensiones requeridos por esta implementación.
Línea 5	using Microsoft.EntityFrameworkCore; → Importa tipos o extensiones requeridos por esta implementación.
Línea 6	using Microsoft.Extensions.Configuration; → Importa tipos o extensiones requeridos por esta implementación.
Línea 7	using Microsoft.Extensions.DependencyInjection; → Importa tipos o extensiones requeridos por esta implementación.
Línea 8	`` → Separa bloques lógicos.
Línea 9	var configuration = new ConfigurationBuilder() → Calcula y conserva el resultado que será validado o mostrado.
Línea 10	.SetBasePath(AppContext.BaseDirectory) → Participa directamente en el flujo validado del checkpoint: .SetBasePath(AppContext.BaseDirectory)
Línea 11	.AddJsonFile("appsettings.json", optional: false) → Participa directamente en el flujo validado del checkpoint: .AddJsonFile("appsettings.json", optional: false)
Línea 12	.AddEnvironmentVariables() → Participa directamente en el flujo validado del checkpoint: .AddEnvironmentVariables()
Línea 13	.Build(); → Participa directamente en el flujo validado del checkpoint: .Build();
Línea 14	`` → Separa bloques lógicos.

Línea 15	<code>var connectionString = configuration.GetConnectionString("AceriaDB");</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 16	<code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");</code> → Participa directamente en el flujo validado del checkpoint: <code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");</code>
Línea 17	<code>``</code> → Separa bloques lógicos.
Línea 18	<code>var services = new ServiceCollection();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 19	<code>services.AddAceriaInfrastructure(connectionString);</code> → Participa directamente en el flujo validado del checkpoint: <code>services.AddAceriaInfrastructure(connectionString);</code>
Línea 20	<code>services.AddScoped<BuenasPracticasUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 21	<code>services.AddScoped<AnalisisSqlUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 22	<code>services.AddScoped<TrackingUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 23	<code>services.AddScoped<IdentityResolutionUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 24	<code>services.AddScoped<NMasUnoUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 25	<code>services.AddScoped<SolucionesNMasUnoUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 26	<code>services.AddScoped<OverFetchingUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 27	<code>services.AddScoped<TraduccionConsultasUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 28	<code>services.AddScoped<SplitQueriesUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 29	<code>services.AddScoped<CompiledQueriesUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 30	<code>services.AddScoped<PaginacionUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 31	<code>services.AddScoped<DiagnosticoRendimientoUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 32	<code>``</code> → Separa bloques lógicos.
Línea 33	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true, ValidateScopes = true });</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 34	<code>using var scope = provider.CreateScope();</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 35	<code>``</code> → Separa bloques lógicos.
Línea 36	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve una dependencia obligatoria desde el ámbito.
Línea 37	<code>context.Database.EnsureDeleted();</code> → Reinicia la base de demostración para un E2E determinista.
Línea 38	<code>context.Database.Migrate();</code> → Aplica la historia real de migraciones heredada.

Línea 39	<code>DemoData.Seed(context);</code> → Carga el dataset determinista de <code>AceriaData</code> .
Línea 40	<code>context.ChangeTracker.Clear();</code> → Limpia tracking antes de la demostración.
Línea 41	<code>` `</code> → Separa bloques lógicos.
Línea 42	<code>var useCase = scope.ServiceProvider.GetRequiredService<DiagnosticoRendimientoUseCase>();</code> → Resuelve una dependencia obligatoria desde el ámbito.
Línea 43	<code>useCase.Ejecutar();</code> → Participa directamente en el flujo validado del checkpoint: <code>useCase.Ejecutar();</code>
Línea 44	<code>` `</code> → Separa bloques lógicos.
Línea 45	<code>Console.WriteLine("4.11 OK");</code> → Publica evidencia observable en la consola.

Paso 7: Compilar

```
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

El build debe finalizar sin errores; el delta se propaga a los estados posteriores.

Paso 8: Ejecutar en LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

POWERSHELL

La salida debe terminar con 4.11 OK. Las aserciones internas fallan si la evidencia no coincide.

Paso 9: Diagnóstico técnico

Reto resuelto. Definir qué métrica distinguiría roundtrips de materialización.

No uses solo tiempo: revisa SQL/roundtrips, cardinalidad, columnas, tracking y frontera de materialización.

Paso 10: Cierre acumulativo

1. Confirma el marcador E2E.
2. Comprueba que `EnsureCreated` no aparece.
3. Conserva las migraciones heredadas.
4. Verifica que `Application` no depende de EF Core.
5. Compara con el checkpoint anterior y documenta el delta.

Errores comunes revisados

- Confundir una medición aislada con una conclusión de rendimiento.
- Materializar antes de terminar filtros o proyecciones sin intención.
- Aplicar una técnica por regla general en lugar de observar la consulta.
- Relajar una aserción para ocultar un fallo en vez de corregir su causa.

Trazabilidad con la práctica fuente

La práctica fuente de 4.11 incluía además los siguientes focos docentes:

- `LogTo`, categorías, niveles, `ILoggerFactory`, `SensitiveDataLogging`, `DetailedErrors` y `ConfigureWarnings`.
- Tiempo, número de comandos, filas y tracking como métricas observables.
- `DiagnosticSource/DiagnosticListener` y detección de consultas lentas.

Tratamiento en el M4 definitivo. La fuente propone un DiagnosticObserver. La baseline validada usa LogTo + DbCommandInterceptor + TagWith para contar comandos y correlacionar consultas de forma determinista. DiagnosticSource se conserva en teoría y como ampliación, no se elimina silenciosamente.

Reto de ampliación procedente de la fuente. Diseña un observador de consultas lentas con un umbral configurable y explica qué aporta frente al interceptor de conteo.

Errores de la fuente que deben seguir siendo diagnosticables

- No incrementar contadores manualmente dentro del repositorio.
- No habilitar SensitiveDataLogging indiscriminadamente en producción.
- No usar una única métrica temporal como diagnóstico completo.

Analogía operativa

Diagnosticar es instrumentar la línea antes de cambiar la máquina.

Resultado esperado

El checkpoint 4.11 queda ejecutable, trazado y reproducible, y sirve como baseline física del punto siguiente.

Conexión con el siguiente punto

El siguiente estado es 4.12 y parte físicamente de este checkpoint.

Punto 4.12 — Estrategias de optimización y checklist de rendimiento

Contexto del proyecto

Este checkpoint continúa M04/PROYECTO/4.11. Conserva solución, capas, filtros, índices y migraciones; M4 no crea migraciones vacías.

Objetivo práctico

El checklist no impone una clasificación universal. Primero se define la forma necesaria, después se observa SQL, roundtrips, materialización y tracking, y solo entonces se eligen o descartan técnicas.

Paso 1: Abrir el checkpoint

```
cd M04/PROYECTO/4.12
dotnet restore AceriaData.sln
```

POWERSHELL

Paso 2: Comprobar migraciones

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

POWERSHELL

Debe seguir apareciendo M2_2_12_Architecture.

Paso 3: Identificar el delta docente

El archivo principal del delta es src/AceriaData.Infrastructure/Repositories/Rendimiento412.cs. El checkpoint conserva todo el estado anterior.

Paso 4: Implementar y estudiar Infrastructure

```
using AceriaData.Application.Dtos;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;
```

CSHARP

```

public sealed partial class OrdenRepositorio
{
    public ChecklistRendimientoDto EjecutarChecklistFinalM4()
    {
        _context.ChangeTracker.Clear();
        SqlCommandCounterInterceptor.Instance.Reset();

        var consulta = _context.OrdenesFabricacion
            .TagWith("M4.12-CHECKLIST-FINAL")
            .AsNoTracking()
            .Where(o => o.Estado == "Pendiente")
            .OrderBy(o => o.FechaCreacion)
            .ThenBy(o => o.Id)
            .Select(o => new OrdenPaginaDto
            {
                Id = o.Id,
                NumeroOrden = o.NumeroOrden,
                Cliente = o.Cliente,
                Estado = o.Estado,
                FechaCreacion = o.FechaCreacion
            })
            .Take(5);

        var sql = consulta.ToQueryString();
        var filas = consulta.ToList();

        return new ChecklistRendimientoDto
        {
            Filas = filas.Count,
            ConsultasSql = checked((int)SqlCommandCounterInterceptor.Instance.Count),
            EntidadesRastreadas = _context.ChangeTracker.Entries().Count(),
            Sql = sql,
            DecisionCompiledQuery =
                "No se aplica: esta consulta final no se ha demostrado como hot path; medir antes de introducir
complejidad.",
            DecisionLoading =
                "Proyeccion escalar: no se cargan navegaciones, por lo que Include/SplitQuery no aportan valor en
esta consulta."
        };
    }
}

```

Application no recibe DbContext, IQueryable ni referencias a Microsoft.EntityFrameworkCore.

Paso 5: Implementar el caso de uso

```

using AceriaData.Application.Interfaces;
namespace AceriaData.Application.UseCases;

public sealed class ChecklistRendimientoUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public ChecklistRendimientoUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== 4.12 CHECKLIST FINAL DE OPTIMIZACION ===");
        var r = _unidad.Ordenes.EjecutarChecklistFinalM4();

        if (r.Filas == 0 || r.ConsultasSql != 1 || r.EntidadesRastreadas != 0)
            throw new InvalidOperationException(
                "4.12: el cierre no cumple consulta unica/no-tracking.");
        if (r.Sql.Contains("Observaciones", StringComparison.OrdinalIgnoreCase))
            throw new InvalidOperationException(
                "4.12: la consulta final incurre en over-fetching.");
        if (!r.Sql.Contains("ORDER BY", StringComparison.OrdinalIgnoreCase))
            throw new InvalidOperationException(
                "4.12: falta ordenacion determinista.");

        Console.WriteLine(
            $"Filas={r.Filas} | Consultas={r.ConsultasSql} | Tracking={r.EntidadesRastreadas}");
        Console.WriteLine(r.DecisionLoading);
        Console.WriteLine(r.DecisionCompiledQuery);
        Console.WriteLine(r.Sql);
    }
}

```

CSHARP

Explicación línea a línea — ChecklistRendimientoUseCase.cs

Línea 1	<code>using AceriaData.Application.Interfaces;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	<code>` `</code> → Separa bloques lógicos.
Línea 3	<code>namespace AceriaData.Application.UseCases;</code> → Sitúa el archivo en la capa y espacio de nombres correspondiente.
Línea 4	<code>` `</code> → Separa bloques lógicos.
Línea 5	<code>public sealed class ChecklistRendimientoUseCase</code> → Declara la clase concreta usada por el checkpoint.
Línea 6	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 7	<code>private readonly IUnidadDeTrabajo _unidad;</code> → Declara la dependencia conservada por la instancia.
Línea 8	<code>public ChecklistRendimientoUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;</code> → Constructor del caso de uso e inyección de la unidad de trabajo.
Línea 9	<code>` `</code> → Separa bloques lógicos.
Línea 10	<code>public void Ejecutar()</code> → Define el flujo principal validado por el E2E.
Línea 11	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 12	<code>Console.WriteLine("=== 4.12 CHECKLIST FINAL DE OPTIMIZACION ===");</code> → Publica evidencia observable en la consola.
Línea 13	<code>var r = _unidad.Ordenes.EjecutarChecklistFinalM4();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 14	<code>` `</code> → Separa bloques lógicos.
Línea 15	<code>if (r.Filas == 0 r.ConsultasSql != 1 r.EntidadesRastreadas != 0)</code> → Comprueba una condición contractual del E2E.
Línea 16	<code>throw new InvalidOperationException(</code> → Hace fallar el checkpoint si la evidencia no coincide.
Línea 17	<code>"4.12: el cierre no cumple consulta unica/no-tracking.");</code> → Participa directamente en el flujo validado del checkpoint: "4.12: el cierre no cumple consulta unica/no-tracking.");
Línea 18	<code>if (r.Sql.Contains("Observaciones", StringComparison.OrdinalIgnoreCase))</code> → Comprueba una condición contractual del E2E.
Línea 19	<code>throw new InvalidOperationException(</code> → Hace fallar el checkpoint si la evidencia no coincide.
Línea 20	<code>"4.12: la consulta final incurre en over-fetching.");</code> → Participa directamente en el flujo validado del checkpoint: "4.12: la consulta final incurre en over-fetching.");
Línea 21	<code>if (!r.Sql.Contains("ORDER BY", StringComparison.OrdinalIgnoreCase))</code> → Comprueba una condición contractual del E2E.
Línea 22	<code>throw new InvalidOperationException(</code> → Hace fallar el checkpoint si la evidencia no coincide.
Línea 23	<code>"4.12: falta ordenacion determinista.");</code> → Participa directamente en el flujo validado del checkpoint: "4.12: falta ordenacion determinista.");
Línea 24	<code>` `</code> → Separa bloques lógicos.
Línea 25	<code>Console.WriteLine(</code> → Publica evidencia observable en la consola.

Línea 26	<code>\$"Filas={r.Filas} Consultas={r.ConsultasSql} Tracking={r.EntidadesRastreadas}");</code> → Participa directamente en el flujo validado del checkpoint: <code>\$"Filas={r.Filas} Consultas={r.ConsultasSql} Tracking={r.EntidadesRastreadas}");</code> ;
Línea 27	<code>Console.WriteLine(r.DecisionLoading);</code> → Publica evidencia observable en la consola.
Línea 28	<code>Console.WriteLine(r.DecisionCompiledQuery);</code> → Publica evidencia observable en la consola.
Línea 29	<code>Console.WriteLine(r.Sql);</code> → Publica evidencia observable en la consola.
Línea 30	<code>}</code> → Delimita el bloque sintáctico asociado.
Línea 31	<code>}</code> → Delimita el bloque sintáctico asociado.

Paso 6: Preparar el composition root

CSHARP

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<BuenasPracticasUseCase>();
services.AddScoped<AnalisisSqlUseCase>();
services.AddScoped<TrackingUseCase>();
services.AddScoped<IdentityResolutionUseCase>();
services.AddScoped<NMasUnoUseCase>();
services.AddScoped<SolucionesNMasUnoUseCase>();
services.AddScoped<OverFetchingUseCase>();
services.AddScoped<TraduccionConsultasUseCase>();
services.AddScoped<SplitQueriesUseCase>();
services.AddScoped<CompiledQueriesUseCase>();
services.AddScoped<PaginacionUseCase>();
services.AddScoped<DiagnosticoRendimientoUseCase>();
services.AddScoped<CheckListRendimientoUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true,
    ValidateScopes = true });
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<ChecklistRendimientoUseCase>();
useCase.Ejecutar();

Console.WriteLine("4.12 OK");
```

Explicación línea a línea — Program.cs

Línea 1	<code>using AceriaData.Application.UseCases;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	<code>using AceriaData.ConsoleApp;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 3	<code>using AceriaData.Infrastructure;</code> → Importa tipos o extensiones requeridos por esta implementación.

Línea 4	<code>using AceriaData.Infrastructure.Persistence;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 5	<code>using Microsoft.EntityFrameworkCore;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 6	<code>using Microsoft.Extensions.Configuration;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 7	<code>using Microsoft.Extensions.DependencyInjection;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 8	<code>``</code> → Separa bloques lógicos.
Línea 9	<code>var configuration = new ConfigurationBuilder()</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 10	<code>.SetBasePath(AppContext.BaseDirectory)</code> → Participa directamente en el flujo validado del checkpoint: <code>.SetBasePath(AppContext.BaseDirectory)</code>
Línea 11	<code>.AddJsonFile("appsettings.json", optional: false)</code> → Participa directamente en el flujo validado del checkpoint: <code>.AddJsonFile("appsettings.json", optional: false)</code>
Línea 12	<code>.AddEnvironmentVariables()</code> → Participa directamente en el flujo validado del checkpoint: <code>.AddEnvironmentVariables()</code>
Línea 13	<code>.Build();</code> → Participa directamente en el flujo validado del checkpoint: <code>.Build();</code>
Línea 14	<code>``</code> → Separa bloques lógicos.
Línea 15	<code>var connectionString = configuration.GetConnectionString("AceriaDB")</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 16	<code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");</code> → Participa directamente en el flujo validado del checkpoint: <code>?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");</code>
Línea 17	<code>``</code> → Separa bloques lógicos.
Línea 18	<code>var services = new ServiceCollection();</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 19	<code>services.AddAceriaInfrastructure(connectionString);</code> → Participa directamente en el flujo validado del checkpoint: <code>services.AddAceriaInfrastructure(connectionString);</code>
Línea 20	<code>services.AddScoped<BuenasPracticasUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 21	<code>services.AddScoped<AnalisisSqlUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 22	<code>services.AddScoped<TrackingUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 23	<code>services.AddScoped<IdentityResolutionUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 24	<code>services.AddScoped<NMasUnoUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 25	<code>services.AddScoped<SolucionesNMasUnoUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 26	<code>services.AddScoped<OverFetchingUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 27	<code>services.AddScoped<TraduccionConsultasUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 28	<code>services.AddScoped<SplitQueriesUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.

Línea 29	<code>services.AddScoped<CompiledQueriesUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 30	<code>services.AddScoped<PaginacionUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 31	<code>services.AddScoped<DiagnosticoRendimientoUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 32	<code>services.AddScoped<ChecklistRendimientoUseCase>();</code> → Registra el caso de uso con ciclo de vida Scoped.
Línea 33	<code>``</code> → Separa bloques lógicos.
Línea 34	<code>using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true, ValidateScopes = true });</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 35	<code>using var scope = provider.CreateScope();</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 36	<code>``</code> → Separa bloques lógicos.
Línea 37	<code>var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();</code> → Resuelve una dependencia obligatoria desde el ámbito.
Línea 38	<code>context.Database.EnsureDeleted();</code> → Reinicia la base de demostración para un E2E determinista.
Línea 39	<code>context.Database.Migrate();</code> → Aplica la historia real de migraciones heredada.
Línea 40	<code>DemoData.Seed(context);</code> → Carga el dataset determinista de <code>AceriaData</code> .
Línea 41	<code>context.ChangeTracker.Clear();</code> → Limpia tracking antes de la demostración.
Línea 42	<code>``</code> → Separa bloques lógicos.
Línea 43	<code>var useCase = scope.ServiceProvider.GetRequiredService<ChecklistRendimientoUseCase>();</code> → Resuelve una dependencia obligatoria desde el ámbito.
Línea 44	<code>useCase.Ejecutar();</code> → Participa directamente en el flujo validado del checkpoint: <code>useCase.Ejecutar();</code>
Línea 45	<code>``</code> → Separa bloques lógicos.
Línea 46	<code>Console.WriteLine("4.12 OK");</code> → Publica evidencia observable en la consola.

Paso 7: Compilar

```
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

El build debe finalizar sin errores; el delta se propaga a los estados posteriores.

Paso 8: Ejecutar en LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

POWERSHELL

La salida debe terminar con 4.12 OK. Las aserciones internas fallan si la evidencia no coincide.

Paso 9: Diagnóstico técnico

Reto resuelto. Auditar una consulta y justificar tanto técnicas aplicadas como descartadas.

No uses solo tiempo: revisa SQL/roundtrips, cardinalidad, columnas, tracking y frontera de materialización.

Paso 10: Cierre acumulativo

1. Confirma el marcador E2E.
2. Comprueba que EnsureCreated no aparece.
3. Conserva las migraciones heredadas.
4. Verifica que Application no depende de EF Core.
5. Compara con el checkpoint anterior y documenta el delta.

Errores comunes revisados

- Confundir una medición aislada con una conclusión de rendimiento.
- Materializar antes de terminar filtros o proyecciones sin intención.
- Aplicar una técnica por regla general en lugar de observar la consulta.
- Relajar una aserción para ocultar un fallo en vez de corregir su causa.

Trazabilidad con la práctica fuente

La práctica fuente de 4.12 incluía además los siguientes focos docentes:

- Checklist, ciclo medir-identificar-aplicar-verificar-documentar y anti-patrones.
- Métricas de tiempo, comandos, volumen, memoria y coste de materialización.
- Estado acumulativo final de AceriaData y documentación de decisiones.

Tratamiento en el M4 definitivo. Se conserva el checklist, pero se corrige la idea de aplicar todas las técnicas a toda consulta. El cierre exige justificar también por qué Include, SplitQuery o CompiledQuery no aplican a una consulta concreta.

Reto de ampliación procedente de la fuente. Audita una consulta completa y documenta cada decisión: aplicada, descartada y evidencia que la sustenta.

Errores de la fuente que deben seguir siendo diagnosticables

- No optimizar antes de medir.
- No forzar todas las técnicas del módulo sobre una misma consulta.
- No considerar una micro-medición aislada como prueba concluyente.

Estado acumulativo real de AceriaData al cerrar M4

- Arquitectura en cuatro proyectos: Domain, Application, Infrastructure y Console.
- Dominio con OrdenFabricacion, PlanchaAcero, Aleacion, EstadoOrden, DetalleOrden, CertificadoCalidad y OrdenAleacion.
- Application mantiene interfaces, DTOs y casos de uso sin depender de Microsoft.EntityFrameworkCore.
- Infrastructure contiene AceriaDbContext, configuraciones Fluent, repositorios, UnitOfWork, migraciones y observabilidad de comandos.
- Modelo con relaciones uno-a-muchos, uno-a-uno y muchos-a-muchos, claves e índices heredados.
- Soft Delete y filtros globales heredados permanecen activos.
- M4 añade análisis SQL, tracking/no-tracking, Identity Resolution, diagnóstico N+1, proyecciones, Split Queries, compiled queries, paginación y métricas.
- Lazy Loading no se activa en M4: los escenarios que podrían producir N+1 se hacen explícitos y medibles.

Analogía operativa

El checklist final no cambia todas las piezas de la máquina, solo las que la medición justifica.

Resultado esperado

El checkpoint 4.12 queda ejecutable, trazado y reproducible, y sirve como baseline física del punto siguiente.

Conexión con el siguiente punto

Este punto cierra el código acumulativo de M4 y consolida el checklist.